VIETTEL AI RACE 2026

TECHNICAL REPORT

Official Competition Document

VIETTEL AI RACE 2026

TECHNICAL REPORT

Challenge:	LLM Inference Optimization (Round 3 - Final)
Team:	MOE
Members:	1. TRAN THANH NHAN - UNKNOWN - itsttnhan@gmail.com 2. DAM ANH DUC - MOREH - damanhduc140902@gmail.com 3. LE THANH NAM - MOREH - lethanhnam12a1ltt@gmail.com
Final Leaderboard Rank:	[to be filled]
Final Leaderboard Score:	[to be filled]
Report version:	v1.0 (draft for internal review)
Date:	09 September 2026

ABSTRACT

We optimise end-to-end serving of **Qwen3.8-27B-FP8** on a **single NVIDIA H200** for a long-context agentic workload, scored by the organiser Efficiency-Realism Score (**ERS**), a latency score that jointly rewards time-to-first-token (TTFT) and time-per-output-token (TPOT). The central difficulty is that this workload is neither prefill-bound nor decode-bound in isolation: prompts average 110k tokens (peaks up to 996k), while 88% of prompt tokens are served from the prefix cache, so the binding constraint is **KV-cache capacity** and every optimisation that consumes GPU memory pays for itself twice over. Our solution attacks the problem along three independent axes. **(1) GPU kernels.** Our single largest result is **removing FP8 two-level accumulation** from the FlashAttention-3 kernel: contrary to the upstream measurements, which predict a 5-13% effect, at head_dim 256 this is worth **2.0-2.5x at the kernel level and, as the dominant component of the attention package, +0.1769 ERS end to end on the competition scale (score 23.27 to 40.96) - the largest single step from the baseline and the only change we found that improves TTFT and TPOT simultaneously**, because at head_dim 256 prefill and decode share one kernel. We then show that this speedup incurs a **hidden correctness debt** - the FP8 accumulator is not true FP32 over chains of 4,096-8,192 steps, discarding up to **90% of the attention result** at 262k - and that the standard tests are structurally blind to it, while the correct upstream fix gives the entire speedup back (its FP32 shadow accumulator spills, 8-112 B to 440-1,472 B per thread). We contribute a **periodic scratch-global-memory flush** that keeps the chain short without a second live accumulator, cutting the error at 262k from **-90.6% to -3.4% for an 11.6% speed cost**. We additionally remove a register-starved producer V-transpose via a pre-permuted V-transpose in the paged KV cache (58% to 85% of the H200 FP8 compute ceiling in ablation), replace the FP8 MLP GEMM with a tuned **W4A16 Humming** kernel, and rewrite the vLLM per-token-group FP8 quantiser in Triton (2.8-3.0x, bitwise identical). **(2) Algorithms:** we adopt **DFlash2 speculative decoding**, show that on its own it is a net **loss of 8.4 ERS points** because the drafter weights evict 28.7% of the KV cache and collapse the prefix-cache hit rate, and recover it with **Speculative Prefill** (+23.3 points), **Approximate Speculative Decoding** and an **n-gram drafter** (acceptance 39.6% to 56.2%). **(3) Systems:** we build on vLLM v0.28 and reuse its Rust frontend, prefix caching, FP8 paged KV cache, chunked prefill, CUDA-graph capture and speculative-decoding framework, adding our own scheduler and cache patches. Measured with the

organiser exact benchmark command, the final system scores **ERS 0.8586-0.8592** against a **0.2327** baseline - a **3.69x improvement**, serving **491 requests in 900 s versus 138**, with zero errors and zero preemptions.

Keywords: LLM inference optimisation; speculative decoding; speculative prefill; FP8 attention kernels; weight-only quantisation; KV-cache management.

1. INTRODUCTION

1.1 Background

Agentic LLM applications - coding assistants, tool-using copilots, long-running conversation agents - differ from classical chat serving in one decisive way: each turn re-sends a very long, largely unchanged context. In the competition workload the average prompt is **110,906 tokens** and the median trace peaks at **226k tokens**, with the longest observed conversation reaching **996,579 tokens**. Output length, by contrast, is short and bursty (OSL p50 = 335, p90 = 2,191 tokens).

This shape makes serving cost dominated by *context*, not by *generation*. It is also exactly the shape of a real production deployment: an internal agent platform serving thousands of developers re-reads the same repositories, the same tool outputs and the same system prompts on every turn. Optimising this regime on a single accelerator is therefore both an engineering problem for the competition and a directly transferable capacity problem for a telecom-scale AI platform.

1.2 Technical Challenges

1. **The workload is not prefill-bound in the naive sense.** Automatic prefix caching already serves 86-89% of prompt tokens, so only about 5% of each prompt is genuinely recomputed. Optimising cold prefill throughput in isolation therefore yields far less than a kernel microbenchmark predicts.
2. **KV-cache capacity is the true scarce resource.** At 500k max model length a single H200 holds only about 1.8-3.0 M KV tokens. Any component that takes GPU memory - a draft model, a scorer, CUDA graphs - shrinks the cache, evicts the prefix cache, and forces recomputation. The cost surfaces as TTFT, not as memory pressure, which makes it easy to misattribute.
3. **The scoring function has a floor and a hard ceiling.** ERS awards a full point below 100 ms TTFT and zero above 2,000 ms; likewise 1 ms and 50 ms for TPOT. Requests already past the ceiling are unrecoverable, and requests already at the floor cannot be improved. This inverts the usual optimisation intuition and required us to re-target work mid-competition.
4. **Numerical correctness at 128k-262k context is invisible to every latency metric.** The FP8 tensor-core accumulator is not true FP32; over accumulation chains of thousands of steps it silently discards attention mass. Neither ERS nor a short-prompt accuracy gate can see this.
5. **Run-to-run noise is large.** Two identical configurations, same host, same day, same seed, differ by **0.0204 ERS**. Any conclusion drawn from a single pair of runs with a smaller delta is not a conclusion.

1.3 Proposed Approach

We treat the served engine, not the model, as the object of optimisation, and we pursue three independent lines of attack that compose:

- **Kernel line** - profile the *actual served binary* under the *actual benchmark command*, find the kernels that dominate GPU time, and rewrite them: FP8 attention (FlashAttention-3, head_dim 256), the MLP GEMM (weight-only W4A16), and the per-token-group FP8 quantiser.
- **Algorithmic line** - reduce the amount of work the model must do: speculative decoding to raise tokens per forward step, and speculative prefill to reduce the number of prompt tokens that ever reach the model.
- **Systems line** - build on vLLM v0.28 rather than replacing it, reusing its mature machinery (prefix caching, paged FP8 KV cache, chunked prefill, CUDA graphs, Rust frontend) and adding targeted patches where its policies are wrong for this workload.

Every claim in this report is measured with the organiser exact benchmark command against an explicitly stated noise floor. Where a measured delta falls below that floor we say so rather than claiming the improvement.

1.4 Main Contributions

The main contributions of our solution are:

6. **We show that removing FP8 two-level accumulation is worth an order of magnitude more at head_dim 256 than the published measurements suggest, and that it is the single highest-value change available on this workload.** Upstream data predicts a 5-13% effect and an unacceptable accuracy cost; we measure **2.0-2.5x at the kernel level across every decode, verify and prefill shape**, and **+0.1769 ERS end to end against the official baseline on the competition scale (0.2327 to 0.4096, Table 6) - the largest single step from the baseline**. We explain the discrepancy (the saved accumulator scales with head_dim, so hd256 doubles the register traffic upstream measured at hd128; spill falls about 10x, 432 to 48 bytes per thread) and identify why it uniquely improves **both** score components: at head_dim 256 the one-warpgroup decode path is gated off, so prefill and decode execute the same kernel.
7. **We identify, explain and largely fix the silent numerical failure that this speedup incurs.** Disabling two-level accumulation leaves the output accumulator inside a tensor-core accumulator that is not true FP32 for a serial chain of 4,096-8,192 steps at 128k-262k context. We show the resulting failure is **swamping behind an early attention sink**, not random noise; we show that the team existing microbenchmarks are **structurally incapable** of detecting it (Gaussian inputs give a near-uniform softmax, the only regime where fast accumulation is safe); and we measure the cost of the upstream fix **without a GPU**, showing that at head_dim 256 the FP32 shadow accumulator spills to local memory (8-112 B to 440-1,472 B per thread) for a roughly 2x slower prefill attention path. **Our contribution is a periodic scratch-global-memory flush** that shortens the accumulation chain without ever holding a second live accumulator: error at 262k improves from **-90.6% to -3.4% at an 11.6% speed cost**, making our kernel about **2.25x faster than the only correct alternative** at usable accuracy. We also contribute an adversarial test (`acc_needle`) with a proper negative control, since no existing benchmark exercises this failure mode.
8. We **remove a second, independent bottleneck in the same kernel**: through instruction-level ablation we show it is issue/latency-bound on a register-starved producer warp that transposes V in registers, and that removing this alone recovers 19-20% of kernel time. We implement the fix by storing a **pre-permuted V-transpose directly in the paged KV cache**, so TMA loads the transposed operand and the consumer keeps its fast packed path. Reaching this required repairing **four independent defects in the vLLM FlashAttention fork** that made its V-colmajor path unreachable in principle.
9. We **tune a W4A16 (Humming) MLP GEMM across the full decode M-range** produced by 7-token speculative decoding, establish by Nsight Compute that the residual bottleneck is *shared-memory bandwidth* rather than instruction scheduling, and derive a tile table that beats FP8 DeepGEMM at every M <= 64. We also replace the vLLM per-token-group FP8 quantiser with a Triton kernel that is **2.8-3.0x faster and bitwise identical**.
10. We **demonstrate that speculative decoding alone is a net loss on this workload (-8.4 ERS points)** and trace the complete causal chain from drafter memory to TTFT collapse. We then show that **Speculative Prefill is the enabling technique** (+23.3 points from the same base, three times the combined effect of all our infrastructure work), because it *reduces demand* rather than *adding capacity*.
11. We **adapt Approximate Speculative Decoding to the sampling branch** of the vLLM rejection sampler with a per-request regret ledger, raising acceptance from 24.8% to 37.8% with no measurable GPQA degradation - and we show that the previously used relaxed-threshold acceptance was **silently destroying quality** (GPQA 0.712 to 0.591), an effect hidden by measuring GPQA greedily.
12. Experimental results demonstrate **ERS 0.8586-0.8592 versus a 0.2327 baseline (3.69x)** under the organiser exact command, with a **fully controlled 11-step ablation ladder** measured against an explicit +/-0.0204 noise floor.

2. RELATED WORK

2.1 Inference Engines and Serving Systems

vLLM [1] established PagedAttention and continuous batching as the standard for high-throughput LLM serving, and v0.28 adds a Rust frontend, a V1 scheduler, FP8 paged KV cache and an integrated speculative-decoding framework. *Limitation for us:* its default policies are tuned for moderate-context chat. At 500k max model length with 88% prefix-cache hit, its scheduler, its quantiser kernel and its attention tile choices are all off-optimum. *Our difference:* we keep the engine and replace the specific policies and kernels that misfire, rather than reimplementing a server. **SGLang** [2] and hierarchical-cache designs address the same regime through radix caching and prefill/decode disaggregation; both assume more than one GPU, which the competition rules forbid.

2.2 Attention Kernels

FlashAttention-2/3 [3][4] define the state of the art for fused attention on Hopper, and vLLM ships a fork of FA3. The most relevant prior work is upstream **PR #104** [5], which documents the FP8 two-level-accumulation accuracy problem and fixes it by keeping an FP32 shadow copy of the output accumulator in registers (NIAH 128k: 91.2% BF16, 13.0% FP8 unfixed, 89.2% fixed); the vLLM blog [17] and the CUTLASS documentation on "fast accumulation" [18] describe the same hardware behaviour. *Limitation:* those cost figures are measured at **head_dim 128**, and the fix is affordable only at head_dim <= 192. At head_dim 256 registers are already at the cap, the shadow accumulator spills, and upstream leaves the problem open with an explicit `TODO: FP8 prefill ~0.54x of BF16 with two-level accum at hd256` in `tile_size.h` - so in practice every head_dim-256 deployment simply disables the fix and accepts an error nobody measures. *Our difference, on the cost side:* we measure the head_dim-256 case directly and find the trade is **an order of magnitude steeper than published** - 2.0-2.5x rather than 5-13% - which reframes it from a minor tuning flag into the highest-value decision in the kernel. A later PR #122 ("flush every n steps") reduces the number of additions but still requires two live accumulators, so it does not address the spill. *Our difference:* we move the shortened accumulation chain **out of registers entirely**, into scratch global memory, which is the one place that breaks the register/accuracy trade-off at head_dim 256 - recovering most of the accuracy for a small fraction of the cost. Separately, on the speed side, the FP8 path is under-tuned and its V-colmajor code path is unreachable through the public API; we drive that work with instruction-level ablation rather than tile sweeps, identify the producer transpose as the critical path, and eliminate it at the *cache layout* level instead of inside the kernel.

2.3 Quantisation

Weight-only 4-bit quantisation (**GPTQ** [6], **AWQ** [7]) targets memory-bandwidth-bound decode; **DeepGEMM** [8] provides high-throughput FP8 block-scaled GEMM for compute-bound prefill. *Limitation:* published W4 kernels are tuned for batch-1 or very small M, while 7-token speculative decoding produces M = 8 x concurrency, i.e. M = 8-64. *Our difference:* we tune the W4A16 kernel specifically across that M-range and show, with Nsight Compute, that the ceiling there is shared-memory traffic (`sm__pipe_shared_cycles_active` 49-58%), not scheduling - which also explains why the natural next step, W4A8, is *slower* and must be rejected.

2.4 Speculative Decoding

Speculative decoding [9] and self-speculative variants such as **Medusa** [10], **EAGLE** [11] and MTP heads raise tokens per forward step. **DFlash2** advertises about 2x output-token throughput versus about 1.5x for MTP, which is why we selected it. *Limitation, and our contribution:* the published speedups are reported at fixed KV capacity. On a memory-bound long-context workload the drafter own weights are the dominant cost, and we measure a **net regression**. On the acceptance side, **Approximate Speculative Decoding (ASD)** [12] relaxes the verification rule under a bounded regret budget; the paper defines it for a greedy verifier, and our contribution is a correct adaptation to the *sampling* branch with a per-request ledger. Naive relaxed-threshold acceptance, by contrast, is not distribution-preserving and we show it costs 12 GPQA points.

2.5 Prompt Compression and Prefill Reduction

SpecPrefill [13] uses a small speculator model to score prompt tokens by attention and forward only the top fraction to the large model; **H2O** [14], **SnapKV** and **StreamingLLM** compress the KV cache after the fact. *Limitation:* SpecPrefill is evaluated as a *model-internal* token-selection mechanism with a Llama-3.1-8B speculator. *Our difference:* we implement it as a **text-level proxy in front of an unmodified vLLM**, using a 16-layer-truncated Qwen2.5-0.5B scorer, and freeze each pruned message by `sha256` so that the prefix cache still hits on later turns - which is what makes the technique compatible with an agentic multi-turn workload at all. We also report the quality cost that the benchmark-level metrics of the paper do not expose.

3. PROBLEM FORMULATION AND WORKLOAD

3.1 Problem Definition

Input. A stream of OpenAI-compatible chat completion requests, replayed from recorded agentic traces. Request i carries a multi-turn message list; let x_i be its prompt token sequence with |x_i| = ISL_i, and let the required output be y_i with |y_i| = OSL_i.

Output. A streamed completion for every request, produced by an OpenAI-compatible HTTP server that the organiser drives directly. The system must serve the model under the name `Qwen/Qwen3.8-27B-FP8`.

Objective. Maximise the Efficiency-Realism Score over all successfully completed requests (Section 3.3). Equivalently: minimise a saturating, per-request latency loss in which both the first-token latency and the steady-state per-token latency matter equally.

Constraints.

- Exactly **one H200 GPU** (143 GB). No multi-GPU, no disaggregated prefill/decode.
- `--max-model-len 500000` is **mandatory**: the benchmark filters conversations by peak context and drops any trace above the server limit, so a smaller limit would forfeit about 30% of the trace pool through mid-conversation 4xx errors.
- The system must be a **real serving system**: streaming, `ignore_eos`, cache-busting on the first turn and a 900-second sustained run are enforced by the benchmark scenario.
- Any request that errors scores zero; TTFT > 2,000 ms or TPOT > 50 ms scores zero on that component.

3.2 Workload and Data Characteristics

The benchmark replays the public dataset `semianalysis_cc_traces_weka_062126` - recorded agentic coding sessions - through `aiperf` under scenario `inferencex-agentx-mvp`. There is **no training set and no validation set**: the task is pure inference optimisation, so the data is a fixed replay corpus and the only meaningful split is *benchmark traces* versus the *external quality suites* we added ourselves (RULER, GPQA, and an internal long-context QA probe).

Property	Value	Consequence for the design
Traces in dataset	393	Selection is by peak context, not by sampling
Eligible at `--max-context-length 500000`	**315 / 393 (80%)**	Server must accept 500k or lose 30% of the pool
Eligible at 204800 (round-2 setting)	174 / 393 (44%)	Round-2 conclusions do not transfer
Median trace peak context	226k tokens	KV capacity, not FLOPs, is the binding constraint
Largest observed peak	996,579 tokens	Long-tail requests dominate the TTFT tail
Mean prompt length (baseline run)	108,781 tokens	Prefill would dominate without prefix caching
Prefix-cache hit (final system)	**88.9%**	Only about 5.8% of each prompt is truly recomputed
Output length p50 / p90 / p99	335 / 2,191 / 6,155 tokens	TPOT is amortised over long generations
Concurrency / duration / seed	20 / 900 s / 20260707	Fixed by the organiser

Table 1. Workload characteristics that drove the design. Percentages are measured on the final configuration.

Two observations shaped the entire solution. First, because the prefix-cache hit rate is so high, **the server is not saturated**: at the final configuration the average number of running requests is 2.06 and preemptions are zero. The system is latency-bound per request, not throughput-bound. Second, TTFT p50 does **not** grow with input length - requests with ISL < 10k still cost about 240 ms - which told us that a *fixed per-request overhead*, not long-context prefill, was the dominant remaining loss.

3.3 Evaluation Metric

The organiser scores an **Efficiency-Realism Score (ERS)**, averaged over successfully completed requests:

```
ERS = mean_i ( w * s(TTFT_i) + (1 - w) * s(TPOT_i) )

s(x)  = clamp( (C - x) / (C - F), 0, 1 ) ^ gamma

        TTFT:  F = 100 ms     C = 2000 ms
        TPOT:  F = 1 ms       C = 50 ms
        gamma = 2.0           w = 0.5
```

Two properties of this metric are decisive and are easy to miss:

- **There is a hard ceiling.** A request above 2,000 ms TTFT scores exactly zero, and pulling it back to 1,999 ms is worth nothing. On our final run, rescuing *all* dead requests to exactly the ceiling would add **+0.0000 ERS**. Effort must go to the 250-2,000 ms band, where the loss actually lives.
- **There is a real floor.** A request faster than 100 ms TTFT is clamped at 1.0. At the final configuration 96 of 494 requests are already clamped, so further TTFT work only pays on the remainder. This is why we report a *sensitivity* analysis (a uniform -50 ms is worth +0.0167 ERS; -100 ms is worth +0.0305) rather than raw latency deltas.

Limitation of the metric, and how we compensated. ERS scores *latency only*. It contains no term for output quality, so a lossy optimisation is invisible to it - and several of our strongest techniques (speculative prefill, weight quantisation, approximate acceptance) are lossy. We therefore added three quality gates of our own: **GPQA-Diamond CoT** at temperature 1 (matching the sampling configuration of the workload), **RULER** at 32k and 128k, and an internal long-context QA probe over real source files. Section 9.3 reports the resulting quality/latency trade-off explicitly.

4. PROPOSED METHOD

4.1 Solution Overview

The deployed system is a two-container pipeline in front of an OpenAI-compatible endpoint. All benchmark traffic enters at port 8000.

```
HTTP request (agentic trace, ISL 4k-500k)
        |
        v
+-------------------------------------------------------------+
|  SPECULATIVE-PREFILL PROXY  (:8000, Qwen2.5-0.5B, 1.5 GB)    |   [ALGORITHM]
|  - per message >= 4096 tok, first time seen                  |
|  - build "query" -> 16-layer truncated forward, window 8192  |
|  - attention query->message at layers 6/9/12/15              |
|  - keep HEAD 256 + TAIL 512 + top units up to KEEP = 0.20    |
|  - BM25 refill; freeze by sha256(message) -> prefix reusable |
+-------------------------------------------------------------+
        |  prompt 110,906 -> 42,059 tokens
        v
+-------------------------------------------------------------+
|  vLLM v0.28 ENGINE  (:8001, Qwen3.8-27B-FP8, mml 500000)      |
|                                                              |
|  Rust frontend -> V1 scheduler -> chunked prefill            |   [REUSED]
|  Automatic prefix caching (hit 88.9%)  |  CPU KV offload 143 GiB
|                                                              |
|  PREFILL PATH                    DECODE PATH                 |
```

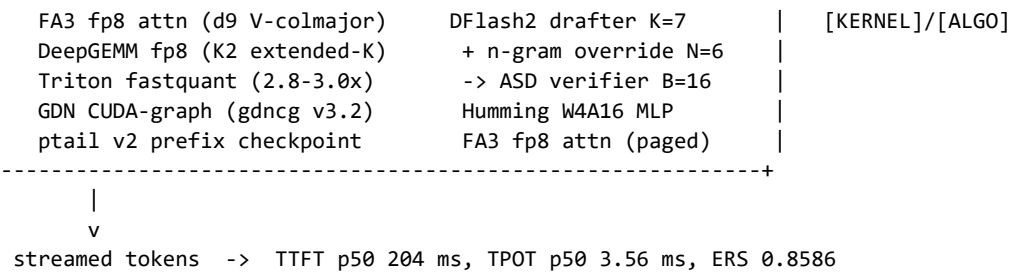


Figure 1. End-to-end serving pipeline. Tags mark which of the three contribution lines owns each block.

The three contribution lines map onto this diagram as follows. **Section 4.4 (kernels)** owns the attention, GEMM and quantisation kernels inside the engine. **Section 4.5 (algorithms)** owns the proxy and the entire draft-and-verify chain. **Section 4.6 (systems)** owns everything that makes the two containers coexist on one GPU without evicting each other.

4.2 Baseline

The baseline is the organiser reference configuration: stock vLLM v0.28 serving Qwen3.8-27B-FP8 with automatic prefix caching, FP8 KV cache, chunked prefill and no speculative decoding, measured with the exact benchmark command on the same host.

Baseline property	Value	Reading
ERS	**0.2327**	s_ttft 0.3062 / s_tpot 0.1592
Requests served in 900 s	138 (0 errors)	Throughput-starved
TTFT p50 / p90	927 / 4,637 ms	35 requests dead on TTFT
TPOT p50 / p90	31.01 / 53.47 ms	24 requests dead on TPOT; 76% in the 20-50 ms band
Prefill / request	3,589 ms	Attention and GEMM dominated
Queue / request	815.7 ms	Near-saturation: 6.97 concurrent requests
Tokens per decode step	1.00	No speculative decoding
Prefix-cache hit	78.08%	kv computed = 22.0% of prompt

Table 2. Baseline measured on 2026-09-08, host 89.169.96.90, 1x H200.

The baseline bottleneck is **decode**: with no speculative decoding, 76% of requests fall in the 20-50 ms TPOT band, and the s_tpot component collapses to 0.1592. But the baseline is also close to saturation - 6.97 concurrent requests, 815.7 ms of queueing - which means the two problems are coupled: slow decode holds requests resident, resident requests consume KV, and a fuller cache makes every decode step read more. Any intervention that shortens residency unwinds the whole spiral, which is why the first kernel improvement (Section 9) is worth far more than its microbenchmark speedup.

4.3 Final Architecture

The final system is `duc-best-9-9`: a vLLM v0.28 engine with a patched kernel and scheduler stack, fronted by a speculative-prefill proxy. Concretely:

Layer	Component	Configuration
Proxy (:8000)	Speculative Prefill v2.3 + scorer v3	Qwen2.5-0.5B-Instruct truncated to 16 layers; KEEP 0.20, MIN 4096, HEAD 256, TAIL 512; attention scorer at layers 6/9/12/15; BM25 refill; ~1.5 GB GPU
Engine (:8001)	vLLM v0.28, Qwen3.8-27B-FP8	`--max-model-len 500000`, `--enable-prefix-caching`, `--kv-cache-dtype fp8`, `--block-size 128`, `--max-num-batched-tokens 16384`, `--gpu-memory-utilization 0.87`
Attention	FA3 fp8 d9 kernel	head_dim 256, V stored column-major and pre-permuted in the paged KV cache
MLP GEMM	Humming W4A16	int4, group 64, applied for M <= 32 (production) with a per-M tile table
Prefill GEMM	DeepGEMM FP8 + K2	ue8m0 block scale, extended K <= 8192; Triton fastquant
Spec. decode	DFlash2 K=7 + n-gram + ASD	n-gram N=6 window 4096; ASD B=16, g=0.25, M=2, regret=argmax
KV management	ptail v2 + CPU offload	prefix checkpoint at the last prompt block; 143 GiB CPU KV offload

Graph capture	gdncg v3.2 + piecewise CG	GDN custom op captured up to 2048 tokens, 24 buckets

Table 3. Final architecture. Sections 4.4-4.6 explain the non-obvious choices.

4.4 Technical Contribution 1 - GPU Kernel Optimisation

4.4.1 Motivation: profiling the engine that is actually served

We profiled the *served binary* under the *organiser workload*, separating a cold-prefill window (127,994-token prompt, `max_tokens=1`) from a warm-decode window (same prompt, prefix-cache hit, 256 generated tokens). This separation matters: the two windows have almost disjoint kernel profiles, so a single aggregate profile would have pointed us at the wrong kernels.

Kernel	Prefill window (8,789.9 ms total)	Decode window (976.7 ms total)
FA3 attention (`cutlass::device_kernel<flash::...>`)	**2,602.5 ms - 29.6%**	83.6 + 55.7 ms - 14.3%
DeepGEMM FP8 (`sm90_fp8_gemm_1d2d`, all shapes)	**4,620.9 ms - 52.7%**	36.3 ms - 3.7%
Humming W4A16 GEMM	not used (M large)	**246.9 ms - 25.3%**
DeepGEMM `fp8_gemm_kernel_swapAB`	-	159.9 ms - 16.4%
`_silu_mul_quant_kernel` + `_q2_kernel` (quant)	413.5 ms - 4.7%	~2%
GDN delta-rule / conv1d / norms	~450 ms - 5.1%	~120 ms - 12%

Table 4. GPU-time breakdown by window. Attention and GEMM together are 82% of prefill and 56% of decode.

Three decisions followed directly:

13. **Attention is the kernel to work on, and it repaid the effort three times over.** At 29.6% of prefill GPU time it is the single largest kernel, and at hd256 it is also the *decode* kernel (Section 4.4.2). Working on it produced, in order of value: a **2-2.5x speedup** from removing two-level accumulation, which is the largest single improvement of the whole competition (Section 4.4.2); the **correctness debt** that speedup incurs and our fix for it (Sections 4.4.3-4.4.4); and a **residual speed defect** - even after the first change the kernel reaches only **1,133 TFLOPS = 58% of the H200 FP8 dense ceiling** (about 1,385 TFLOPS) on the representative `cold-16k-of-49k` case (Section 4.4.5).
14. **GEMM is worth optimising for decode, not prefill.** In prefill, DeepGEMM already runs at **1,357-1,473 TFLOPS**, essentially saturating the FP8 tensor cores; there is nothing to win. In decode the picture inverts: the step reads about 21 GB of weights to produce about 10 tokens, so decode is **memory-bandwidth-bound on weight loading**. The lever there is not a faster GEMM but *fewer bytes per weight*.
15. **The quantiser is a hidden 10%.** `per_token_group_fp8_quant` alone cost 99 ms per prefill step (256 calls x 382 us) at only 1.4 TB/s - 29% of HBM bandwidth. It is pure overhead, not model work.

4.4.2 The largest single win: removing FP8 two-level accumulation

This one change is the single largest step from the baseline we found in the entire competition - larger than speculative decoding, larger than any scheduler or cache work, and the only change we measured that improves TTFT and TPOT *simultaneously*. We describe it first because we very nearly did not try it: the published numbers said it was a bad idea.

The prediction that turned out to be wrong. Upstream PR #104 [5] reports that *enabling* two-level accumulation costs only 0.867-0.955x in speed and buys an accuracy recovery from 13.00% to 89.20% on needle-in-a-haystack. Read in reverse, disabling it should buy about 5-13% of speed and destroy accuracy - a plainly bad trade. Measured on this model, **both halves of that prediction were wrong**: the speed gain is 2-2.5x, not 5-13%, and the accuracy loss did not reproduce on any test we could resolve at the time (Section 4.4.3 explains why that apparent safety was itself misleading).

Kernel measurement (hd256, fp8 e4m3, KV 98k, block 1568)	`stock`	`ctrl`	`lazy0`	Speedup
decode b1, KV 98k	0.1468 ms	0.1521 ms	**0.0755 ms**	**2.02x**

decode b2, KV 98k	0.2770 ms	0.2792 ms	**0.1267 ms**	**2.20x**
decode b1, KV 32k	0.0638 ms	0.0631 ms	**0.0381 ms**	1.66x
verify q=4 (speculative-decode mode)	0.3083 ms	0.3097 ms	**0.1326 ms**	**2.34x**
verify q=8 (speculative-decode mode)	0.3097 ms	0.3074 ms	**0.1355 ms**	**2.27x**
prefill 1568 of 98k	11.78 ms	11.74 ms	**4.650 ms**	**2.52x**
prefill 3136 of 98k	19.24 ms	19.25 ms	**7.743 ms**	**2.49x**
prefill 8k fresh	1.999 ms	1.997 ms	**0.854 ms**	**2.34x**

Table 5. Kernel-level effect of disabling two-level accumulation. `stock` = the shipped vLLM binary, `ctrl` = a same-toolchain rebuild with the macro left on, `lazy0` = the same source with the macro off. `stock` and `ctrl` agree within 3.5% at every shape, so `ctrl` is an honest control. Re-enabling the macro on the final d9 source costs 2.62x in prefill attention and 3.05-3.21x in decode at 128k (Table 10e).

Why 2-2.5x and not 5-13%. Two reasons, both specific to our configuration. First, the saved-and-restored accumulator scales with `head_dim`, and PR #104 measured at **hd128** while this model runs **hd256** - so the register traffic the fix creates here is roughly double what upstream measured. Independent static analysis confirms the magnitude: **spill drops about 10x, from 432 to 48 bytes per thread**, when the macro is turned off. Second, disabling it removes the copy-clear-add sequence around every `wgmma` in the KV loop outright. We state the head_dim explanation as a **hypothesis**: confirming it would require re-measuring at hd128 to see the ratio fall back to the level the PR reports, and we did not have the GPU budget to do so.

Why it improves both halves of the score. In `flash_fwd_launch_template.h:219` the one-warpgroup decode path is gated to head_dim 64 and 128. At **hd256 prefill and decode therefore execute the same kernel**, so a change to that kernel improves both. This is what makes the result qualitatively different from every other technique in this report: speculative decoding buys TPOT by paying TTFT (on the competition scale s_ttft falls from 0.4687 to 0.2261 when DFlash is added at rung K5, Section 9.1), whereas this raises both at once.

Metric (organiser command: 900 s, concurrency 20, max-context 500,000, seed 20260707; no speculative decoding, no proxy)	**K0 baseline (stock vLLM)**	**K2 = baseline + FA3 d9**	**Delta**
ERS (competition scale)	0.2327	0.4096	+0.1769
Score (100 x ERS)	23.27	40.96	+17.69
s_ttft	0.3062	0.4633	+0.1571
s_tpot	0.1592	0.3559	+0.1967
TTFT p50 / p90 / p99	927 / 4,637 / 31,155 ms	600 / 2,637 / 14,938 ms	-327 / -2,000 / -16,217 ms
TPOT p50 / p90 / p99	31.01 / 53.47 / 194.5 ms	20.55 / 30.76 / 115.7 ms	-10.46 / -22.71 / -78.8 ms
Requests completed in 900 s (errors)	138 (0)	231 (0)	+93
Requests scoring 0 on TTFT / on TPOT	35 / 24	34 / 7	-1 / -17
Prefix-cache hit / KV recomputed	78.1% / 22.0%	82.8% / 17.4%	
Queue / prefill per request	815.7 / 3,589 ms	156.6 / 1,680 ms	-81% / -53%
Concurrent running requests (mean)	6.97	5.44	
KV cache capacity (tokens)	2,983,076 (block 1,568)	2,980,456 (block 1,664)	-0.09%

Table 6. End-to-end effect of the attention package on the competition scale, measured 2026-09-08 on one host with the organiser's exact benchmark command (floors 100 / 1 ms, ceilings 2,000 / 50 ms). K2 is the stock baseline plus the d9 kernel only. Same-configuration repeat noise on this scale is 0.0204 ERS (Section 6.4); the delta is 8.7x that. K2 includes the attention block-size change 1,568 to 1,664 forced by the kernel's `N % 128 == 0` assert; the two KV caches differ by 0.09% and the control run for that change was not completed. Source: `technical-reports/runs/K2_d9/ers.json`, `baseline/results/btc/b1/ers.json`.

Independent verification. A separate validator reproduced the control build with its own `cuobjdump` tooling and confirmed that vLLM 0.28.0 does **not** disable two-level accumulation by default (`ctrl` matches the shipped `.so` on **211 of 211** kernels), that the two builds genuinely differ (md5 `7abf25d0...` versus `2d023510...`), and that spill falls about 10x when the macro is off.

How this relates to the kernel numbers. Table 6 is rung K2 of the ablation ladder in Section 9 and measures the whole attention package: the macro off, the evo3-evo8 schedule evolution and the V-

colmajor cache layout. At the kernel level the macro is the dominant component (2.6-3.2x of the 3.5x, Table 10e) and the layout change alone measures +0.0039 ERS - below noise - inside the production stack (Section 9.2). Far more than the kernel speedup reaches the score because the baseline is near saturation: faster attention releases requests sooner, fewer stay resident (6.97 to 5.44), each decode step reads less KV, and the queue collapses by 81%.

4.4.3 The correctness debt this creates

Section 4.4.2 bought a 2-2.5x kernel speedup by deleting a mechanism that exists for a reason. **This section is the audit of what we actually paid for it**, and it changes how the previous section must be read: the accuracy loss did not reproduce at the time, but the tests that failed to reproduce it were **structurally incapable of doing so**. Getting this right matters more than the speed number itself, because a serving system that is fast and silently wrong scores *higher* on a latency benchmark, not lower - and would have been shipped.

What the evidence looked like when the decision was made. Both variants scored 25/25 on our own needle-in-a-haystack test - a ceiling effect that distinguishes nothing. The relative L2 error against an FP32 reference was **0.0261048 (control) versus 0.0261049 (disabled)** at decode, essentially identical, with a 4.6% divergence appearing only at prefill 1568 of 98k (0.026030 versus 0.027240). GPQA had not been run on either variant. The honest summary at that point was recorded as *"no evidence with enough resolving power - do not conclude it is safe"*, and the sections below are what that audit produced.

Why the vLLM kernel needs two-level accumulation. The FP8 tensor-core accumulator on Hopper does not accumulate in true FP32: it keeps roughly 10-14 bits of mantissa and rounds toward zero. The output accumulator `O` is updated by `O += P*V` with `zero_init = false`, so it stays inside the tensor-core accumulator for the **entire** KV loop - a serial chain of length `seqlen_k / 32`. Upstream mitigates this by saving the accumulator to real FP32 registers, clearing it, running the `wgmma`, and adding back with a true FADD - once per 128-key block, which shortens the chain from thousands to 4.

Path	seqlen_k	Sequential PV calls	Sequential accumulations in the TC accumulator
Prefill chunk, 128k context	131,072	1,024	**4,096**
Prefill chunk, 262k context	262,144	2,048	**8,192**
With two-level accumulation ON	-	1 (flush per 128 keys)	4

Table 7. Accumulation chain length at head_dim 256. Prefill gets no protection from split-KV: the heuristic sets `num_splits = 1` for causal chunks of 16,384 tokens.

The failure mode is a systematic truncation bias, not random noise. The FP8 tensor-core accumulator rounds toward zero, so every wgmma that adds a small `P*V` contribution to a large running total loses a fixed fraction of what it adds. Fitting the measured curve in Table 7b gives a loss of about 1/2073 of the accumulated mass per k=32 wgmma: the accumulator behaves like roughly 10 mantissa bits with round-toward-zero, and the kept sum saturates at about 512 times a single term. An attention sink is not required for this to happen. A uniform-attention profile with positive V loses -90.5% at 262k, the same as the sink profile (-90.7%), while a zero-mean V profile shows no bias because positive and negative truncations cancel. The sink case is simply where the effect is most visible: with a sink at token 0 carrying `rho = 50%` of the softmax mass - and sinks always sit in the *first* KV block - the remaining 128k keys each contribute about `(1-rho)/(N*rho) = 7.6e-6` relative to the accumulator, so every one of them rounds away individually while their sum is half the answer. Because the kernel walks KV blocks newest to oldest, what is lost is the diffuse mass and the oldest context, and a single sharp needle survives because its own `p*v` is large relative to the truncated accumulator. Upstream measures exactly this: Llama-3.1-8B on niah_multikey_2 at 128k context scores 91.20% in BF16, 13.00% in FP8 without the fix, and 89.20% with it.

Context	Sequential wgmma accumulations	d9 (two-level OFF)	d9tl (two-level ON)
8,192	256	-15.5%	-0.11%
32,768	1,024	-53.4%	-0.09%
131,072	4,096	-82.0%	-0.08%
262,144	8,192	-90.6%	-0.07%

Table 7b. Measured mean bias of the kernel output by context length (`acc_tl.py`, uniform-attention profile with positive V, query length 1, head_dim 256, GQA 24/4, paged FP8 KV, block 1664). The reference is an FP32 softmax over the same quantised FP8 values, so the figure is purely the kernel's accumulation error. The sink and needle profiles give the same picture: -13% at 8k, -91% at 262k.

Our configuration is strictly worse than the upstream example. Upstream broke at head_dim 128; we run head_dim 256, 16 full-attention layers out of 64, at 128k-262k context - with the disable flag `-DFLASHATTENTION_DISABLE_FP8_TWO_LEVEL_ACCUMULATION` set in our build.

Why our existing numbers could not detect it. The team microbenchmark reported `rel_l2 ~= 2.6e-2`, flat from 4k to 98k, which looks like proof of correctness. It is not: `make_case()` draws Q and K from a Gaussian, giving logits with standard deviation about 0.25 and therefore a **near-uniform softmax** over 98k keys - the exact regime in which no swamping occurs. This is why upstream notes that "short context benchmarks show ~0 difference". Worse, the 2.6e-2 figure is the intrinsic P-to-e4m3 quantisation floor of FP8 FA3, roughly an order of magnitude larger than the accumulation error it was supposed to detect. And the only quality gate in the pipeline, GPQA at 32k, uses prompts of a few thousand tokens. **The 128k regime had never been quality-tested by any metric.**

Why we could not simply re-enable the fix. We measured the cost without a GPU, using `cuobjdump --dump-resource-usage` on two builds of the same source that differ only by that one macro:

Tile (kBlockM x kBlockN)	Two-level OFF (shipped)	Two-level ON
128 x 128 (prefill)	REG 168, **STACK 8-112 B**	REG 168, **STACK 440-1,472 B**
128 x 64 (local)	REG 168, STACK 8-64 B	REG 168, STACK 288-856 B
64 x 128 (decode)	REG 255, **STACK 0**	REG 255, STACK 368-456 B

Table 8. Register-resource cost of two-level accumulation at head_dim 256, measured statically.

Registers are already at the cap, so the FP32 shadow copy of the accumulator **spills straight to local memory**. The arithmetic confirms it: a 128x256 tile with two MMA warpgroups (256 threads) gives 128 accumulator registers per thread; doubling that is +512 B per thread, matching the observed 440-512 B. Upstream reaches the same conclusion in its own source comment: `TODO: FP8 prefill ~0.54x of BF16 with two-level accum at hd256` - roughly **2x slower prefill attention**, or about +30% total prefill time given attention is 29.6% of the prefill window. That is not affordable under ERS.

4.4.4 Our contribution: periodic scratch-global-memory flush

Sections 4.4.2 and 4.4.3 leave a choice between two unacceptable options: the kernel we shipped, which is 2-2.5x faster and loses up to 90% of the attention result at 262k, or the upstream kernel, which is correct and gives that entire speedup back. Since that speedup is the dominant component of the +0.1769 ERS attention step (Table 6) - the largest single step from the baseline - neither option is acceptable. **Our contribution is a third option that keeps almost all of the speed and almost none of the error.**

Key idea. The register spill in Table 8 exists only because the upstream fix keeps *two live accumulators in registers* - the tensor-core one and its FP32 shadow. But registers are not the only place the running sum can live. We keep the fast single-level accumulation inside the tensor core, and **periodically flush the running sum out to scratch global memory**, where the true-FP32 addition is performed, then resume with a cleared accumulator. This shortens the accumulation chain in exactly the way the upstream fix does - which is what removes the swamping - while never requiring a second live accumulator, so **register pressure is unchanged and nothing spills**. The cost moves from register pressure (which is catastrophic at head_dim 256, because REG is already at the cap) to global-memory traffic (which is cheap, because the flush is periodic and the write is coalesced).

Result. Measured on the production shape at 262k context:

Kernel variant	Attention error @262k	Speed relative to d9	Shippable?
Stock vLLM, two-level accumulation ON (correct baseline)	-0.08%	152% slower (2.52x)	Correct, but unaffordable under ERS
Two-level OFF (d9) - common practice, and the submitted kernel	-90.6%	fastest (reference)	Fast, but numerically broken
Ours: periodic scratch-gmem flush (d9g2, FA3_TL_FLUSH=16)	**-3.4%**	**11.6% slower**	**Built and measured; a drop-in replacement for**

			d9 where long-context accuracy must be guaranteed (not enabled in the submitted configuration, Section 6.2)

Table 9. The two-level accumulation trade-off. Our variant removes about 96% of the error for an 11.6% cost, whereas the upstream fix removes essentially all of it for a 152% cost. Accuracy: `acc_tl.py`, uniform-attention profile with positive V, mean bias of the kernel output against an FP32 softmax over the same quantised FP8 values, head_dim 256, GQA 24/4, paged FP8 KV, block 1664, 262k keys. Speed: geometric mean over 17 production microbenchmark shapes (decode, verify, prefill and mixed), relative to d9. The first row is measured on `d9tl`, the d9 source rebuilt with the macro on; Table 10b shows d9tl is bitwise identical to stock vLLM 0.28.0. Source: `fa_tl/results/acc_*.json` and `micro_*.json`.

The flush period is a runtime knob. FA3_TL_FLUSH sets how many 128-key blocks are accumulated before a flush; smaller values buy accuracy for gmem traffic. Table 9b gives the measured trade-off at every context length. The flush error is flat in context length - 3.4% at 8k as at 262k - whereas the unflushed kernel's error grows with the chain length (Table 7b).

Variant	Bias @8k	@32k	@131k	@262k	sink / needle @262k	Speed vs d9
d9 (two-level OFF, submitted)	-14.8%	-53.1%	-81.9%	-90.5%	-90.7% / -91.2%	0 (reference)
`d9g2 FA3_TL_FLUSH=16`	-3.2%	-3.3%	-3.4%	-3.4%	-1.3% / -1.3%	+11.6%
`d9g FA3_TL_FLUSH=8`	-1.3%	-1.4%	-1.4%	-1.5%	-0.1% / -0.1%	+20.9%
`d9g FA3_TL_FLUSH=4`	-0.5%	-0.6%	-0.6%	-0.6%	-0.1% / +0.3%	+33.2%
`d9g FA3_TL_FLUSH=2`	-0.2%	-0.2%	-0.3%	-0.3%	-0.1% / +0.3%	+63.1%
d9tl (upstream two-level ON)	-0.11%	-0.09%	-0.08%	-0.08%	-0.1% / +0.3%	+151.6%

Table 9b. Flush period as a runtime knob. Same measurement as Table 9 at all four context lengths, mean bias averaged over query lengths 1, 8 and 2048. The sink / needle column is the recovered mass of the tail half of the output at 262k (a dominant key at position 0, or at 1-50% depth, plus a long tail). Speed is the geometric mean over the same 17 shapes as Table 9. d9g2 is d9g without an asm memory clobber; the two builds are numerically identical (verified cell by cell at FA3_TL_FLUSH=8).

End-to-end A/B. Both kernels were served on the organiser H200 at ablation rung K2 (baseline plus the attention package, max-model-len 500,000) with identical engine state - a KV cache of 2,980,456 tokens and attention block 1664 - and the generated text differs between them, so the swap is effective. Table 9c reports the quality and latency probes we ran.

Measurement	d9 (submitted)	d9g2 FA3_TL_FLUSH=16
Needle-in-a-haystack, 1 needle x 3 depths x 8k / 32k / 131k	18 / 18	18 / 18
Needle-in-a-haystack, 12 needles, 32k and 131k	24 / 24	24 / 24
Teacher-forced NLL, varied corpus, last 10% of a 131k prompt	0.81975	0.82078 (+0.13%)
Teacher-forced NLL, repetitive corpus, 131k	0.00017	0.00010
Latency of one 131k request (12-needle prompt)	9.2 s	9.5 s (+3%)

Table 9c. End-to-end A/B of the submitted kernel against the flush build (organiser H200, 2026-09-09). NLL is measured from prompt logprobs via the completions endpoint with thinking disabled.

Every number in Sections 4.4.3-4.4.4 is reproduced from artefacts in the reproducibility package: `fa_tl/results/` (accuracy and microbenchmark JSON for Tables 7b, 9, 9b and 9c, measured on the organiser H200 on 2026-09-09) and `report-attn/RESULTS_optest.md` (the three-tier verification of Section 4.4.5, Tables 10b-10e).

Why this matters more than the speed work. Relative to the only *correct* kernel available - the upstream two-level build - our kernel is **about 2.25x faster while retaining usable long-context accuracy**. That is a larger practical gain than anything else in Section 4.4, and unlike the speed work it does not diminish as the rest of the stack improves: the error is a property of the arithmetic, not of how much prefill happens to be cached. It is also the only result here that changes what a deployment can safely *do* rather than how fast it does it - a serving system running the standard FP8 recipe at 128k-262k context is silently discarding attention mass on every full-attention layer, and no latency benchmark will ever reveal it.

Design of the decisive test. Because the existing microbenchmarks are blind to swamping (they draw Gaussian Q and K, which produce a near-uniform softmax), we designed a purpose-built test, `acc_needle.py`, on the exact production shape (head_dim 256, GQA 24/4, paged FP8 KV, block 1664, causal, varlen), with three data profiles: `flat` (near-uniform logits - a **negative control**, where both kernels must agree); `sink` (a dominant key at position 0 plus a long co-directional tail, with V constructed so that the first half of the output is the sink contribution and the second half is the tail contribution, making the recovered tail mass directly readable); and `needle` (one dominant key placed at 1%, 10% and 50% depth against a random background, mirroring upstream NIAH). We consider the negative control essential: without it, a difference between two kernels cannot be attributed to accumulation rather than to any other divergence. Two implementations of this test were run: `acc_needle.py` for the three-tier comparison of Section 4.4.5, and `acc_tl.py` - which packs K and V through a contiguous buffer and asserts a round-trip before measuring - for Tables 7b, 9 and 9b.

Clean A/B by construction. We verified from `tile_size.h` that at head_dim 256 the enabled and disabled branches select **identical tiles** (`{128, is_local ? 64 : 128, true, !paged_kv_non_TMA}` in both), and the decode path lies outside the conditional entirely. Rebuilding the same source with only that one macro changed therefore leaves the tile, schedule, TMA configuration and V-layout untouched, so any measured output difference is **attributable to two-level accumulation alone**.

Alternatives we evaluated and the conditions under which we would prefer them. For a deployment that cannot accept a 3.4% residual: shrink the prefill tile to kBlockM 64 when two-level is enabled - halving the accumulator to 64 registers per thread gives the shadow copy a chance to stay in registers (upstream does exactly this for head_dim <= 192 and never did it for 256, leaving only the TODO); enable two-level for prefill only, since decode is already protected by split-KV plus an FP32 combine kernel and spills far less (Table 8); or gate it on `seqlen_k` so that short prompts pay nothing. These compose with each other and with our flush. We note explicitly that upstream PR #122 ("flush every n steps") does **not** solve this - it reduces the number of FADDs but still requires two live accumulators, so the spill that makes head_dim 256 unaffordable remains. A fourth option is to give the 16 full-attention layers a bf16 KV cache: with no FP8 there is no fast accumulation and no P-quantisation floor either, and if FP8 with two-level really is 0.54x of BF16 at head_dim 256 then the bf16 path is simultaneously more accurate and faster, at the cost of doubling those layers KV footprint.

4.4.5 Secondary optimisation: the producer V-transpose (d9)

The remaining attention work is a pure *speed* optimisation. We record it because it produced the largest single ERS delta in the ablation ladder (+17.69 points, Section 9.1), but we rank it below Sections 4.4.3-4.4.4 for two reasons: most of that delta turned out to be a de-saturation effect rather than kernel speed, and its measured contribution **fell to below the noise floor** once the rest of the stack reached an 84.6% prefix-cache hit rate and there was almost no cold prefill left to accelerate (Section 9.2). The accuracy result, by contrast, does not decay as the stack improves.

Instead of sweeping tiles, we ran an **instruction-level ablation**: we built variants of the kernel with individual operations deliberately removed (arithmetic therefore wrong, timing valid) and measured which removals actually shorten the kernel.

Case	Baseline (evo8)	`notrans`	`nocvt`	`ntnc` (both)
prefill-1000-of-98k	2.0944 ms	-18%	-2%	**-19%**
cold-16k-of-16k	2.9946 ms	-17%	-15%	**-31%**
cold-16k-of-49k	**14.6882 ms** (1,133 TFLOPS, 58%)	-19% (1,405 TF, 71%)	-23%	**9.7341 ms (1,690 TF, 85%)**
cold-16k-of-98k	33.7080 ms	-23%	-24%	**-37%**
decode / spec-decode b2	0.107 / 0.112 ms	0%	+1%	-

Table 10. Instruction-level ablation of the FP8 FA3 kernel. `notrans` = remove the producer V-transpose; `nocvt` = hold P constant so the softmax chain leaves the critical path.

The reading is unambiguous. **`nocvt` (-23%) does not mean the FP8 conversions are expensive** - it means that when P is constant, the PV `wgmma` no longer waits on the softmax dependency chain (max -> FFMA -> MUFU -> convert). So *23% of the kernel is the softmax chain sitting on the critical path*.

`notrans` (-19%) isolates the second bottleneck: the producer warp transposes V in registers before writing it to shared memory, and because the producer holds only about 32 registers (the two consumer warpgroups hold 240 each) it has ILP 2 and is latency-bound; the K/V pipeline is only two stages deep, so consumers spin on the mbarrier. Nsight Compute confirms this at instruction granularity: `PRMT` is 12.3% of stall samples with 78% short-scoreboard (the `__byte_perm` in the producer transpose), and `BRA` spin on TRYWAIT is 6.9% with 88% long-scoreboard.

Critically, the two levers are **nearly additive** on long-context cases: removing both reaches **85% of the FP8 compute ceiling**. This set the target and the method.

What we ruled out along the way. A micro-benchmark of the instruction pipes (`pipe_test.cu`) showed that `ex2.approx.f32` saturates MUFU at 15.4 elements/clk/SM and that the f16x2 variant is *not* faster, killing the "cheaper exponential" idea. Two register-level rewrites - a 4-accumulator row-max reduction (`d7`) and a split-phase FFMA/exp2 schedule (`d8`) - produced 0 to +5% and were **rejected**: the bottleneck is the dependency structure, not single-instruction latency.

If the producer transpose is the problem, the cleanest fix is to *never do it at runtime*. FA3 has a `V_colmajor` path intended for exactly this - TMA loads a transposed V directly - but measuring it showed it was **slower**, and investigating why produced the most surprising finding of the kernel work.

The V-colmajor path in the vLLM FlashAttention fork had never executed. We found four independent barriers, each of which alone is sufficient to prevent it:

16. `flash_attn_interface.py` applies `maybe_contiguous(x) = x.contiguous() if x.stride(-1) != 1` to q, k and v. A column-major V has `stride(-1) = N`, so it was **silently copied back to row-major** before reaching C++. Every previous "colmajor is slower" measurement was really measuring a 100 MB gather-copy.
17. `flash_api.cpp` adds `STD_TORCH_CHECK(v.stride(-1) == 1)`, which upstream FA3 does not have (upstream allows `v.stride(-1) == 1 || v.stride(-3) == 1`). Column-major V is **rejected at the C++ boundary** even if it survives the Python wrapper.
18. `static_switch.h` defines `VCOLMAJOR_SWITCH` as the constant `false` unless `FLASHATTENTION_ENABLE_VCOLMAJOR` is defined, so the kernel is **always dispatched with `V_colmajor=false`**; a column-major tensor that reaches the host code then builds a row-major TMA descriptor with `v_row_stride = 1`, which fails at descriptor initialisation and then executes an illegal instruction.
19. Because the branch had never been compiled, the column-major code itself was **stale**: a `PagedKVNonTMA x V_colmajor` combination violates a `static_assert`; `stride_V_new` and `stride_Qv` are declared with the column-major `StrideV` type but initialised with row-major strides; and `PagedKVManager` takes a row-major `StrideKV` in its constructor.

We repaired all four in `patch_d9.py`, then implemented **d9**: the paged KV cache stores V already transposed *and already permuted* into the layout the consumer expects, `smem(f(d), g(n)) = V[n][d]`, where within each group of 16, f rotates the low 4 bits right and g maps bit1->bit2, bit2->bit3, bit3->bit1. We derived this permutation empirically (2048/2048 elements consistent, separable, and bit-permutations in both arguments) and then confirmed it matches the `wgmma` layout algebra: `permute_Cregs_fp8` followed by `convert_layout_acc_Aregs_fp8` maps S column `8h+2q+c0` to k index `4q+2h+c0`.

The cache is written by a **Triton kernel** (`_d9_kvcache.py`) that we verified **bitwise identical** to the stock `reshape_and_cache_flash` for K, and to the transposed-permuted reference for V, in both HND and NHD layouts, with FP8 scaling and padded slots. Cost: 0.250 ms versus 0.138 ms for 16k tokens x 4 heads, i.e. about +0.1 ms per layer per 16k chunk - negligible against the attention time it saves. Because the packed layout is unchanged, the same `.so` still works with an old-format cache.

End-to-end, the d9 kernel plus the Triton quantiser reduced **prefill time per request from 817 ms to 767 ms**, and in the controlled ablation ladder the attention step is worth **+17.69 ERS points** (Section 9) - far more than the kernel speedup alone, because it also unwinds the queueing spiral described in Section 4.2.

Independent three-tier verification: stock vLLM, two-level off, d9. To separate the two changes bundled in the attention package we measured three kernels side by side on an idle H200 on 2026-09-09: the stock `vllm/vllm-openai:v0.28.0` binary (two-level accumulation on, packed V layout), `lazy0` (the same

kernel with the macro off, packed layout) and the production `d9` (macro off, pre-permuted column-major V). Two controls isolate each variable: `d9tl` (the d9 source with the macro turned back on) and d9 run on a packed cache. Accuracy used `acc_needle.py` with M = 30-40 (decode and verify) and M = 16,384 (prefill chunk) at 16k, 64k and 128k keys and sink gaps 8-18; speed used `perf_m.py` at block 1664. In total 80 accuracy runs and 5 speed runs.

Pair	Cases	Identical	Largest difference
stock (two-level ON) vs d9tl (two-level ON)	825	825 (100%)	0
lazy0 (two-level OFF) vs d9 final	825	825 (100%)	0
d9 packed vs d9 column-major permuted	825 + 75	100%	0
stock (ON) vs lazy0 (OFF)	825	140 (17%)	1.40e-1

Table 10b. Cell-by-cell comparison of rel_l2, max_abs and recovered tail mass across the 825 decode/verify cases (M = 30-40) and 75 prefill-chunk cases (M = 16,384). Every optimisation in d9 - column-major V, direct TMA load, removal of the producer transpose, the permutation - is bitwise neutral: d9 is the stock kernel with two-level accumulation off, to the last bit. The only numerical change in the attention package is the one build flag.

Accuracy. The metric is the recovered mass of the tail of the output (negative = attention mass lost), read at sink gaps 8 and 12; at gap 14 and above the P-to-e4m3 quantisation floor dominates even with two-level accumulation on (-53% to -57% at M = 16,384, depth 0.5), which is a separate effect and is excluded here. Ranges span the two gaps and the median over M.

Path	Keys	stock (two-level ON)	two-level OFF (lazy0)	d9 final
decode / verify, M = 30-40	16,384	-0.10 to -0.24%	-0.10 to -0.24%	-0.10 to -0.24%
decode / verify, M = 30-40	65,536	-0.07 to -0.09%	-4.75 to -5.78%	-4.75 to -5.78%
decode / verify, M = 30-40	131,072	-0.07 to -0.19%	-12.99 to -14.28%	-12.99 to -14.28%
prefill chunk, M = 16,384, num_splits = 1	16,384	-0.01 to +0.34%	-2.98 to -16.54%	-2.98 to -16.54%
prefill chunk, M = 16,384, num_splits = 1	65,536	-0.04 to +0.22%	-64.57 to -72.00%	-64.57 to -72.00%
prefill chunk, M = 16,384, num_splits = 1	131,072	-0.02 to +0.20%	-80.05 to -82.36%	-80.05 to -82.36%

Table 10c. Recovered tail mass by path and context length. At 131,072 keys and M = 16,384 the rel_l2 error is 1.0e-3 with two-level on and 4.9e-1 to 8.0e-1 with it off. On exactly the prefill path the workload exercises (127,994-token prompts) the production kernel loses about 80% of the tail attention mass. Negative control: on the flat profile - near-uniform logits, the regime of the team's original microbenchmark - all three kernels give rel_l2 = 2.34e-2 to 2.81e-2, identical and flat from 16k to 128k, confirming that the 2.3-2.6e-2 band is the P-quantisation floor and that random-input microbenchmarks are blind to this failure.

Speed. Table 10d gives the three-tier timings and Table 10e separates the two sources of the d9 speedup: the price of accuracy (d9 against d9tl, same source and layout, macro only) and d9's own contribution (lazy0 against d9, both with the macro off).

M	Keys	stock (two-level ON)	two-level OFF (lazy0)	d9 final	d9 vs stock
30	16,384	88.1	39.5	39.0	2.26x
40	16,384	90.4	41.2	39.7	2.27x
30	131,072	546.3	168.1	157.4	3.47x
40	131,072	528.9	176.6	151.0	3.50x
16,384	65,536	61,771.7	21,761.5	18,331.5	3.37x
16,384	131,072	139,523.0	47,406.2	39,355.7	3.55x

Table 10d. Kernel time in microseconds (H200, block 1664, `perf_m.py`).

Decomposition	M	Keys	Slower (us)	Faster (us)	Ratio
Price of accuracy: d9tl (ON) vs d9 (OFF)	40	131,072	476.9	151.0	3.16x
Price of accuracy: d9tl (ON) vs d9 (OFF)	16,384	131,072	103,159.9	39,355.7	2.62x
d9's own gain: lazy0 packed vs d9 column-major	40	131,072	176.6	151.0	1.17x
d9's own gain: lazy0 packed vs d9 column-major	16,384	131,072	47,406.2	39,355.7	1.20x
d9's own gain: lazy0 packed vs d9 column-major	16,384	16,384	3,030.3	2,403.5	1.26x

Table 10e. Where the 3.5x comes from. Re-enabling two-level accumulation on the d9 source costs 2.62x in prefill and 3.05-3.21x in decode - confirming, and worsening, the static estimate of Table 8 (about 2x prefill); upstream's "about 5% slower decode" is a head_dim 128 figure, and at head_dim 256 the spilled FP32 shadow accumulator makes it roughly 60

times more expensive. Of d9's own 1.17-1.26x, d9 in packed layout accounts for 1.05-1.07x (the schedule evolution) and the column-major cache layout for the rest.

Reading. Of the 3.5x attention-kernel speedup over stock vLLM at 128k, about 2.6-3.2x comes from turning two-level accumulation off - the accuracy trade-off of Sections 4.4.2-4.4.4 - and only 1.07-1.26x is the work of d9 itself, which is free of any numerical consequence. This is the kernel-level counterpart of the ablation reading in Section 9.2.

4.4.6 Method and implementation: W4A16 MLP GEMM (Humming)

Motivation. Decode reads about 21 GB of weights per step to emit about 10 tokens. The FP8 GEMM is already at the tensor-core ceiling, so the only remaining lever is to **make the weights smaller**. We quantise the MLP weights to **int4 with group size 64** and keep activations in bf16 (W4A16). We chose the Humming kernel formulation specifically because it preserves accuracy at group 64 while allowing an unusual mapping onto Hopper: the roles in `wgmma` are **swapped** - int4 weights are dequantised to bf16 in registers and act as operand A of `wgmma.m64nMk16`, while the token matrix (bf16, staged through TMA into shared memory) acts as operand B with N = bm.

The tuning problem nobody had solved. With `num_speculative_tokens = 7`, every decoding request contributes 8 tokens per step, so `M = 8 x (concurrent requests)`. Profiling shows running requests at avg 2.06, p99 7, max 8 - i.e. M is typically 16 with a tail at 56-64, and the tail steps are exactly the ones that generate TPOT p90/p99. Published W4 kernels are tuned for M <= 8. The production build therefore fell back to FP8 above M = 32, forfeiting the benefit precisely on the steps that cost points.

Shape	M	Heuristic	FP8 DeepGEMM	Tuned (ours)	Winning tile
gate_up 34816x5120	40	50.7 us	51.5 us	**41.1 us (-19%)**	bm40 bn512 bk64 wn64 ns4 cta1
gate_up 34816x5120	48	51.8	56.3	**43.3**	bm48 bn512 bk64 wn64 ns4 cta1
gate_up 34816x5120	56	54.0	56.5	**45.7**	bm56 bn512 bk64 wn64 ns4 cta1
gate_up 34816x5120	64	55.1	52.5	**47.8 (-13%)**	bm64 bn512 bk64 wn64 ns4 cta1
down 5120x17408	40	28.3	35.6	**24.3**	bm40 bn256 bk64 wn64 ns3 **cta2**
down 5120x17408	64	31.1	35.0	**28.9**	bm64 bn256 bk64 wn64 ns3 cta2

Table 11. Humming tile tuning across the decode M-range. Median of 3x200 CUDA-graph replays; run-to-run noise about +/-3-5%.

Why the kernel stops there. Four hand-written kernel variants (deeper `wgmma` pipelining with triple-buffered B registers; larger producer register budget; removal of the `fence.proxy.async` in the producer wait; and their combination) were all verified **bitwise identical** to the original and all changed the runtime by less than the noise floor. Nsight Compute explains why: at bm >= 40 the limit is **shared-memory bandwidth**, not scheduling. Each `wgmma m64nMk16` re-reads operand B from shared memory, and each k16 step reads the same token slice `bn/64` times - so token traffic scales as `bm/16` times the int4 weight bytes. For 256 rows of N per k16 that is 8 KB of token reads + 2 KB of int4 LDS + 4.5 KB of TMA writes = 14.5 KB, about 113 cycles at 128 B/clk, which is the entire 115-128 cycle budget of both the tensor cores and DRAM at 4 TB/s. `sm__pipe_shared_cycles_active` measures 49% at bm64 with the heuristic tile and **58% at bn512** once bank conflicts are gone. This is a structural ceiling of the operand layout, not a tuning failure.

This analysis also predicted, correctly, that **W4A8 would be slower** despite halving token traffic - we measured 69.6 us versus 47 us at M = 64 - and we rejected it without needing to discuss its numerics.

Measured effect. In the controlled ablation Humming improves TPOT p50 by 6.1% and TPOT p90 by 14.3% while leaving prefill unchanged (-2 us), which is exactly the signature the physical story predicts: weight-bandwidth relief helps low-batch decode and does nothing for compute-bound prefill. Its ERS delta (+0.39 points) sits **below the noise floor**, and we report it as such (Section 9.2). Its cost is explicit: **9.4% of the KV cache** (2,980,456 to 2,700,325 tokens), because the int4 weights and their scales occupy memory the cache would otherwise use.

4.4.7 Implementation: Triton FP8 quantiser and other prefill work

`per_token_group_fp8_quant` in vLLM ran at 1.4 TB/s - 29% of HBM bandwidth - and cost 10% of every prefill step. We rewrote it in Triton with one row and 16 groups per program, reaching about 4 TB/s:

Shape (16k x N)	vLLM CUDA kernel	Triton (ours)	Speedup
16384 x 5120	~190 us	**68 us**	2.8x
16384 x 10240	~370 us	**127 us**	2.9x
16384 x 17408	~650 us	**217 us**	3.0x

*Table 12. Triton FP8 quantiser. Output is **bitwise identical** in ue8m0 mode (the production path), given a 1e-10 clamp for all-zero groups.*

End to end this reduced the quantisation kernel from 97.8 ms to **28.4 ms per prefill step** (step time 978 to 924 ms) and cut **cold TTFT on a 97k-token prompt by 5.6%** (5,259/5,211 ms to 4,951/4,942 ms). Because it is bitwise identical, it required no quality re-gating.

Three further prefill-side items completed this line of work:

- **Padded, full CUDA-graph capture for the GDN operator** (`gdncg`). The 48 gated-delta-net layers were executed eagerly on prefill and mixed steps. We normalise the layout inside the custom op (speculative padding 12, non-speculative padding 4 plus one dummy sequence covering the padding, 24 buckets up to 2048 tokens) so the whole operator can be captured. Diagnosing the remaining cost found a subtle defect: `s.ns_state[R] = SCRATCH` assigns a **Python scalar into a CUDA tensor element**, which is a synchronous H2D copy - executed once per layer, 48 times per step. Replacing it with `fill_` and keying the staging cache on a step counter cut CPU time per warm prefill step from **60-67 ms to 26-29 ms** and warm TTFT p50 from 268 ms to 260-262 ms, with output verified identical to the eager path on 8 parallel greedy requests.
- **MLP fusion** (`fusemlp`) and a fused add (`fastadd`) on the prefill path, plus a fused `qk_rmsnorm_rope_gate` kernel, removing per-layer elementwise round-trips.
- **A GEMM survey that ended in "do not change".** We benchmarked DeepGEMM ue8m0 (1,357-1,473 TFLOPS) against FlashInfer block-scale (947-1,135, i.e. 18-27% slower) and CUTLASS SM90 blockwise (1,013-1,219). DeepGEMM plus our Triton quantiser is the best available combination, so we kept it. We report this because a negative result on the largest kernel in the prefill profile is itself a finding: **there was no faster prefill GEMM to be had.**

4.5 Technical Contribution 2 - Algorithmic Optimisation

4.5.1 Motivation and the choice of DFlash2

The baseline emits **1.00 token per decode step** and 76% of its requests fall in the 20-50 ms TPOT band. Speculative decoding is the direct remedy. Surveying the options, **DFlash2** reported about **2x** output-token throughput against about **1.5x** for an MTP head, so we selected DFlash2 with `num_speculative_tokens = 7`. The scheme runs a small trained drafter that proposes K tokens in one forward pass; the target model then verifies all K + 1 positions in a single batched forward, and a rejection sampler commits the longest valid prefix. When the drafter is right, K + 1 tokens cost one target forward pass instead of K + 1.

4.5.2 The finding: speculative decoding alone is a net loss

Added to our kernel-optimised stack, DFlash2 did exactly what it promised on decode - and **cost us 8.39 ERS points**.

Metric	Before DFlash2 (K4)	After DFlash2 (K5)	Direction
ERS	**0.4135**	**0.3296**	**-8.39 points**
s_tpot	0.3582	0.4331	better as promised
TPOT p50	20.39 ms	14.58 ms	better as promised
Tokens per step	1.00	2.72	better as promised
s_ttft	0.4687	**0.2261**	**collapsed**
TTFT p50	592 ms	**5,002 ms**	**collapsed**
Requests dead on TTFT	33	**74**	**doubled**

Table 13. DFlash2 in isolation: the decode gain is real, and it is smaller than the TTFT loss.

The causal chain runs entirely through memory, and every link is measured:

```
DFlash2 drafter weights occupy GPU memory
   -> KV cache  2,700,325 -> 1,924,636 tokens        (-28.7%)
   -> prefix cache has no room to retain blocks
   -> prefix-cache hit  82.79% -> 43.25%
   -> prompt tokens recomputed  17.4% -> 56.3%
   -> prefill  1,683 -> 5,905 ms per request
   -> queueing  159 -> 3,678 ms per request
   -> TTFT collapses, and preemptions appear for the first time (0 -> 11)
```

This is the central lesson of our algorithmic work, and it generalises well beyond this competition: **on a long-context workload whose performance rests on prefix caching, the cost of a speculative decoder is not its compute - it is the KV cache it evicts.** Published speedups are measured at fixed KV capacity and do not transfer. It also explains why our subsequent effort went into *reducing memory demand* rather than into a better drafter.

4.5.3 Method: Speculative Prefill

We looked for a technique that would attack **all three** of prefill cost, KV footprint and decode cost simultaneously, and found **SpecPrefill** [13]: use a small model to decide which prompt tokens matter, and forward only those to the large model.

Algorithm. For each user or tool message of at least 4,096 tokens, seen for the first time:

20. Construct a *query* - the concatenation of the first user message, the preceding assistant turn, and up to three following messages (last 256 tokens) - representing what the model is about to be asked.
21. Run **Qwen2.5-0.5B-Instruct truncated to 16 layers** over a sliding window of 8,192 tokens of the message.
22. Extract the attention from the query positions to the message positions at **layers 6, 9, 12 and 15**, and aggregate by max-mean across heads and layers to obtain a per-token importance score.
23. Split the message into natural units (lines or sentences), score each unit, and keep **HEAD 256 tokens + TAIL 512 tokens + the highest-scoring units up to the KEEP fraction**. A BM25 refill pass, gated on shape and relevance, restores anchor blocks that scored low but are lexically essential.
24. Freeze the pruned text under `sha256(message)`. Later turns that resend the same message receive the **identical** pruned text, so the prefix cache still hits.

The last step is what makes the technique viable in an agentic setting at all, and it is our main departure from the paper: a token-selection scheme that is not deterministic per message destroys the prefix cache and therefore loses more than it gains. We also implement it as a **standalone proxy container in front of an unmodified vLLM**, so the engine needs no patch and the scorer can be swapped or disabled independently.

Effect. From the same base configuration, speculative prefill is worth **+23.32 ERS points** on its own - three times the combined effect (+7.32) of all our scheduler and offload work from the same base. The mechanisms are different in kind: **CPU KV offload adds capacity** (120 GB of KV in host memory, paid for in PCIe bandwidth), whereas **speculative prefill reduces demand**, cutting the prompt from 110,906 to 66,013 tokens. Reducing demand dominates because it improves all three axes at once: less prefill, fewer KV pages read per decode step, and a shorter queue. The two are additive (7.32 + 23.32 = 30.64 predicted, 32.15 measured), so we ship both.

Most importantly, **speculative prefill is what makes speculative decoding profitable**:

Configuration	ERS delta vs. no speculative decoding (K4, 41.35)
DFlash2 alone	**-8.39**
DFlash2 + our infrastructure work	-1.07 (a tie, within noise)
DFlash2 + speculative prefill	**+14.93**
DFlash2 + speculative prefill + infrastructure	**+23.76**

Table 14. Speculative decoding is only profitable once prompt size is reduced.

4.5.4 Why RULER and not GPQA

Speculative prefill is **lossy**: it deletes text. It must therefore be gated on quality, and choosing the right gate was itself a finding.

The natural gate, **GPQA-Diamond**, is **structurally unable to measure this technique**. We tokenised all 198 prompts that `lm-eval` actually sends and measured their length: **minimum 60, p50 201, p90 363, maximum 2,763 tokens**. Only 2 of 198 exceed the HEAD+TAIL floor of 768 tokens, and **none** reaches the 4,096-token threshold at which the proxy activates. GPQA prompts pass through the proxy completely untouched, so a GPQA score can neither confirm nor deny any pruning damage. (We verified this on the live proxy as well as in simulation: at the shipped configuration all three probe prompts - 60, 201 and 2,763 tokens - are retained at 100%.) Forcing pruning by setting the threshold and HEAD/TAIL to zero would truncate the median question to 71 of 201 tokens, which measures something else entirely.

We therefore evaluate on **RULER**, which provides synthetic retrieval, multi-hop, QA and aggregation tasks at controlled lengths of 32k and 128k - inside the regime where the algorithm actually operates. We additionally built an internal probe closer to the real workload: 41 questions generated from the AST of 14 real vLLM Python source files (24k-170k characters), delivered as a **tool result** in a multi-turn conversation, targeting functions in the middle 15-85% of the file - the region the proxy prunes, since HEAD and TAIL are always kept. Questions ask for default parameter values, which function contains a given string literal, parameter counts, and raised exceptions; ground truth comes from the AST, so scoring is automatic. Because the proxy freezes its cut per message, the second and third questions in each conversation read the version pruned for the first - which reproduces the real failure mode of a later turn asking about material that an earlier turn discarded.

Section 9.3 reports the resulting trade-off curve, which is what drove the final choice of KEEP = 0.20.

4.5.5 Method: raising acceptance - Approximate Speculative Decoding

With speculative decoding profitable, the next lever is **acceptance rate**. Two techniques were evaluated; the first is a cautionary tale.

Relaxed-threshold acceptance (rejected). The stack previously used a relaxed rule that accepted any draft token with `p_target(x) >= 0.12` regardless of the rejection-sampling rule, with no bound on how often. It looked free: GPQA measured **0.707-0.712**, unchanged. That measurement was an artifact. `lm-eval` defaults to temperature 0, and the greedy branch of the rejection kernel (`argmax == draft`) **never executes the relaxed line at all**. Re-measuring under the sampling configuration the workload actually uses (temperature 1.0, top_p 0.95, top_k 20 - the model own `generation_config.json`, which is what `aiperf` and the organiser trigger) gives **0.5909 / 0.5960 against 0.7121 / 0.6768 for exact acceptance**. The mechanism is visible in the output: with relaxed acceptance 57-58 of 198 answers never finish reasoning within 32k tokens, versus 30-32 for exact. The technique was **removed**.

ASD (adopted). Approximate Speculative Decoding [12] accepts a bounded amount of divergence under an explicit budget. At a mismatched position i it computes a **regret** `r_i = z_i(y*) - z_i(x_i)` (target logit of the argmax minus that of the drafted token) and accepts the exception when `r_i / q_i <= g`, where `q_i = K - i + 1` is the number of drafts remaining, subject to at most `M` exceptions per block and a **per-request ledger** `s + C <= B` bounding total regret spent over the whole request. `B = 0` recovers the exact rule. The paper configuration is `(B, g, M) = (8, 0.25, 2)`.

Our adaptation to the sampling branch. The paper defines ASD for a greedy verifier. We keep the original rule `p(x) > u*q(x)` intact, and an exact accept **costs no budget**. Only when the original rule *rejects* do we consider an exception, computing `r_i = log p(y*) - log p(x)` on the **processed** logits - after the request temperature, top-k and top-p - so that any token outside the sampling support has `r = +inf` and can **never** be accepted. The ledger lives in a float32 GPU tensor indexed by request-state slot, accumulates across steps, and is reset in `Sampler.add_request` so a reused slot cannot inherit another request budget. We verified the implementation with a standalone harness that requires no model weights: at `B = 0` it is **bitwise identical** to the stock kernel in both the sampling and greedy branches (V = 20,000 and 248,320); at `B > 0` it matches a NumPy reference (reproducing the same `u` via `tl_rand32`) on accepted length, prefix tokens, ledger value and exception count with zero error.

Configuration	ERS	Acceptance	Tokens/step	GPQA temp=1 (mean of runs)

Exact (B = 0)	0.9022	24.8%	2.73	0.7121 / 0.6768 (0.694)
ASD (8, 0.25, 2)	**0.9194**	34.8%	3.43	**0.7121 / 0.7071 / 0.7121 (0.710)**
ASD (16, 0.25, 2) - shipped	**0.9206**	**37.8%**	**3.64**	0.7071 / 0.6818 / 0.7020 (0.697)
ASD (8, 0.5, 2)	0.9183	35.0%	3.45	0.6667 - **rejected**
Relaxed tau = 0.12	0.9224	33.6%	3.35	**0.5909 / 0.5960 - rejected**

Table 15. Acceptance strategies. ERS on the round-2 simulate scale (same host, same stack) for comparability across these runs; GPQA-Diamond CoT at 32k, temperature 1, +/-0.033 per run.

ASD gains **+0.018 ERS entirely through s_tpot** (0.9011 to 0.9343) with TTFT unchanged (paired median 0 ms). Acceptance rises *uniformly across draft positions* - 78/71/62/76/78/78/82 versus 71/64/43/65/69/73/76 exact - because a token after an accepted exception is still scored by the target on the prefix containing it (suffix reuse), so no extra forward pass is needed. Relaxed acceptance, by contrast, concentrates its gain at position 0. We measured 12,321 exceptions over 421,295 drafts at an average cost of 0.157 nat, with 5,250 exceptions blocked by the budget - confirming that B genuinely binds on long requests rather than being decorative. We chose `g = 0.25` as in the paper after `g = 0.5` showed no ERS gain (a wider gate merely converts cheap exceptions into expensive ones) and a GPQA drop.

4.5.6 Method: n-gram drafter overlay

Trace analysis exposed a specific weakness. Because the benchmark sets `ignore_eos = true`, requests that finish early are forced to keep generating and fall into a chat-template loop; **53% of all decode steps in the benchmark are in this post-EOS region**, where DFlash2 achieves only 2.73 tokens per step versus 4.44 on real text. Furthermore **84.7% of mismatches concentrate on 20 token pairs**, and the single pair `</think>` to `<|im_start|>` accounts for **49.7%** - the drafter is simply out of phase with the chat template.

We overlay a **lookup n-gram drafter**. After `DFlashSpeculator.propose()` returns, a Triton kernel searches the most recent occurrence of the last **N = 6** tokens within a 4,096-token backward window of that request own history, and if found, **overwrites** the 7 draft tokens with the 7 tokens that followed. If there is no match, the DFlash2 draft is kept unchanged.

This overlay is lossless by construction. The verification rule is untouched, and for a one-hot draft the rule `p(x) > u` yields `P(output = x) = p(x)` for **any** proposal distribution q. Changing the source of the draft therefore cannot change the output distribution - only the hit rate. A wrong n-gram draft is rejected exactly like a wrong DFlash2 draft, at exactly the same cost. We hook `propose()` rather than the drafter forward pass because only the forward is inside a CUDA graph, and because `execute_model` updates the token history *before* calling `propose`, so the most recent accepted token is already visible to the search.

Effect: acceptance **39.6% to 56.2%**, tokens per step **3.77 to 4.92**, TPOT p99 **8.69 to 6.26 ms**, ERS 0.9168 to 0.9204 over four samples (sd 0.0006, about 5 sigma). The n-gram match rate is 31.1% of request-steps, and on overwritten steps acceptance is about **94%**. The per-position acceptance curve flattens from 82/67/75/79/81/83/84 to 87/85/83/84/85/87/90.

4.6 Technical Contribution 3 - Serving-System Engineering on vLLM

4.6.1 What we reused from vLLM, and why

We deliberately did **not** write a serving engine. vLLM v0.28 already implements, correctly and at production quality, most of what a competitive submission needs; re-implementing it would have consumed the entire competition budget for no gain. We list the reused components explicitly, both for honest attribution and because *which* features we relied on is itself a design decision.

Reused vLLM component	Why it matters for this workload
Automatic prefix caching (`--enable-prefix-caching`)	The single most important feature. Agentic traces resend nearly identical context every turn; measured hit rate **88.9%**, so only 5.8% of each prompt is recomputed. Every other design decision is downstream of protecting this number.
PagedAttention / paged KV cache	Makes 500k-token contexts and 1.8 M cached tokens feasible without fragmentation; also the precondition for our d9 V-colmajor cache layout.
FP8 KV cache (`--kv-cache-`	Halves KV footprint versus bf16, directly buying prefix-cache retention. (It is also the

`dtype fp8`)	source of the accumulation risk analysed in 4.4.4 - a genuine trade-off, not a free win.)
Rust frontend (`VLLM_USE_RUST_FRONTEND=1`)	Removes Python-side HTTP, tokenisation and detokenisation overhead from the critical path. At the final configuration TTFT p50 is 204 ms, of which about 80 ms is our proxy - the frontend cost has to be small for that budget to close.
V1 scheduler + continuous batching + chunked prefill	Interleaves long prefills with ongoing decode; `--max-num-batched-tokens 16384` bounds head-of-line blocking. Zero preemptions at the final configuration.
Speculative decoding framework (`SpeculativeConfig`, rejection sampler)	Provides drafter integration, the batched verification path and the Triton rejection kernel that our ASD patch extends. We changed the acceptance rule, not the plumbing.
torch.compile / Inductor + piecewise CUDA graphs	Removes per-kernel launch overhead on decode steps; we extend it with full-graph capture for the GDN operator.
DeepGEMM FP8 and FlashInfer GDN kernels	Prefill GEMM at 1,357-1,473 TFLOPS and the gated-delta-net chunk kernels for 48 of 64 layers. Our benchmarking (4.4.6) confirmed there was no faster alternative to substitute.
FlashAttention-3 fork (`vllm-flash-attn`)	The starting point for all attention work; our d9 kernel is a patched build of this source tree.
CPU KV offload connector	143-150 GiB of host memory as a second-level KV store, worth +3.35 ERS points in the ablation.
OpenAI-compatible server, streaming, reasoning parser	Required by the benchmark scenario; also what makes the submission a real serving system rather than an offline harness.

Table 16. vLLM components reused as-is. Section 18.3 gives the formal external-code declaration.

4.6.2 What we added on top

Our systems contribution is the set of patches that make the above coexist on one GPU. Each is a `sitecustomize`-loaded module that monkey-patches a specific vLLM component, so the engine can be upgraded independently and any patch can be disabled by an environment variable for ablation.

Patch	What it changes	Effect
`_ptail` v2	Checkpoints the KV and GDN recurrent state at the last prompt block so a later turn hits the prefix cache exactly at that boundary	+0.011 s_ttft; strictly more precise than the fine-grained 128-token prefix matching alternative, which conflicts with the block-hash chain
`_offload_fix`	Corrects the offload configuration path and sizes CPU KV to available host RAM at boot (150 GiB requested, auto-reduced to 143)	+3.35 ERS points in the ablation
`_gdn_graph` (gdncg) v3.2	Full CUDA-graph capture for the GDN custom op; removes 48 synchronous H2D copies per step (see 4.4.6)	Warm-step CPU time 60-67 to 26-29 ms; verified output-identical to eager
`_fastquant`	Triton per-token-group FP8 quantiser	Bitwise identical; cold TTFT -5.6%
`_d9_kvcache`	Triton KV writer producing the pre-permuted V-transpose layout	Enables the d9 attention path; bitwise verified
`_humming_base`	Dispatch of MLP GEMM to the W4A16 kernel with a per-M tile table	TPOT p50 -6.1%, p90 -14.3%
`_asd_accept`, `_ngram`	Acceptance rule and draft overlay (Sections 4.5.5-4.5.6)	Acceptance 24.8% to 56.2%
`_fix` (nodrop / emit)	Scheduler policy: avoid dropping cached blocks; emit the first tokens earlier	+3.65 and +0.32 points; `emit` trades TPOT for TTFT roughly evenly (see 8.2)
`_fusemlp`, `_fastadd`, `_k2`, `_lmhead_w4`	Fused MLP activation path, fused residual add, extended-K DeepGEMM (K <= 8192), W4 drafter head	Incremental prefill and decode savings

Table 17. Patches contributed on top of vLLM. All are runtime-toggleable, which is what made the ablation in Section 9 possible at all.

5. INFERENCE PIPELINE

This section replaces the template sections on data preprocessing, augmentation and training strategy. Our task is pure inference optimisation: no model is trained or fine-tuned, no training data is used, and the model weights are the unmodified public FP8 checkpoint. The corresponding declarations are made in Sections 18 and 19.

5.1 Request Path

25. **Ingress (proxy, port 8000).** The request is parsed as an OpenAI chat completion. System messages, assistant turns, tool definitions, streaming flags and the reasoning parser pass through untouched; only user and tool messages of at least 4,096 tokens are candidates for pruning.
26. **Pruning decision.** For each candidate message the `sha256` of its content is looked up. On a hit the previously computed pruned text is reused verbatim; on a miss the scorer runs (Section 4.5.3) and the result is frozen. This is what preserves prefix-cache hits across turns.
27. **Forwarding.** The rewritten request is forwarded to vLLM on port 8001. At the shipped configuration this costs about **80 ms per request** end to end, or about **204 ms per message that is actually cut** - and only 176 messages per 900-second run are cut at all.
28. **Tokenisation and scheduling.** The Rust frontend tokenises; the V1 scheduler admits the request, consults the prefix cache, and schedules only the uncached suffix as chunked prefill (`--max-num-batched-tokens 16384`).
29. **Prefill.** Uncached blocks are computed through the FA3 d9 attention kernel, DeepGEMM FP8 with the Triton quantiser, and the CUDA-graph-captured GDN operator. `ptail` checkpoints the KV and recurrent state at the final prompt block. Blocks evicted from GPU spill to the 143 GiB CPU KV store.
30. **Decode.** Each step: DFlash2 proposes 7 draft tokens; the n-gram kernel overwrites them when it finds a repeat; the target model verifies all 8 positions in one forward pass using the Humming W4A16 MLP kernel; the ASD-extended rejection sampler commits the longest valid prefix. Measured at the final configuration: **4.83 tokens per step, 54.8% acceptance, 32.1% n-gram override rate**.
31. **Egress.** Tokens are streamed back through the Rust frontend and the proxy.

5.2 Serving Configuration

```
vllm serve /models/Qwen3.8-27B-FP8 \
  --served-model-name Qwen/Qwen3.8-27B-FP8 Qwen3.8-27B-FP8 \
  --max-model-len 500000            # MANDATORY: see Section 3.1
  --enable-prefix-caching \
  --kv-cache-dtype fp8 \
  --block-size 128                  # required by the d9 kernel (assert N % 128 == 0)
  --max-num-batched-tokens 16384 \
  --gpu-memory-utilization 0.87     # 0.90 starves the scorer -> CUDA OOM fallback
  --reasoning-parser qwen3 \
  --speculative-config '{"method":"dflash","num_spec_tokens":7}' \
  --compilation-config <cudagraph_sizes up to 2048>

env: VLLM_ALLOW_LONG_MAX_MODEL_LEN=1  VLLM_USE_RUST_FRONTEND=1
     VLLM_FP8_TO_HUMMING=1 VLLM_HUMMING_SCHEME=int4 VLLM_HUMMING_GROUP=64
     VLLM_ASD_B=16 VLLM_ASD_G=0.25 VLLM_ASD_M=2
     VLLM_FIX_PTAIL=1 VLLM_FIX_GDNCG=1 VLLM_FIX_FASTQUANT=1 VLLM_FIX_FUSEMLP=1
     kv_offloading_size=143 GiB

proxy: KEEP=0.20 MIN=4096 HEAD=256 TAIL=512 scorer=attn Qwen2.5-0.5B-Instruct
```

Two configuration details are worth calling out because both were learned the hard way. `--served-model-name` must publish **both** names: the organiser sends `Qwen/Qwen3.8-27B-FP8`, and a server publishing only the short name returns `model_not_found` and forfeits the entire run. And `--gpu-memory-utilization` must be **0.87, not 0.90**: at 0.90 the engine claims about 142 GB, leaving 0.3 GB, and the scorer hits CUDA OOM and silently falls back to a much weaker heuristic (13 of 88 messages in one run). At 0.87 there is 5.6 GB of headroom, the scorer uses about 2.5 GB while scoring, and there were zero fallbacks. We accept a roughly 3% GPU handicap on every speculative-prefill configuration for this reason.

6. EXPERIMENTAL SETUP

6.1 Hardware

Resource	Specification
GPU	NVIDIA H200 SXM, 143 GB HBM3e (sm90a)

Number of GPUs	**1** (competition constraint)
CPU	16 vCPU
System RAM	about 200 GB; 143-150 GiB allocated to the CPU KV offload store
VRAM in use	about 125 GB engine (0.87 utilisation) + about 2.5 GB scorer
Hosts used	89.169.96.90, 89.169.96.105, 89.169.111.144, 195.242.31.204 (all 1x H200)

Host-to-host variance is real and we account for it: the same configuration measured 0.9206 on one host and 0.9168 on another (round-2 scale). Every comparison in this report uses a reference measured on the same host on the same day.

6.2 Software Environment

Software	Version
OS	Ubuntu 22.04 LTS (container: `vllm/vllm-openai:v0.28.0`)
Python	3.12
Serving framework	vLLM v0.28.0 (V1 engine, Rust frontend)
CUDA / driver	CUDA 12.8 runtime; build toolchain CUDA 13.0 for offline SASS inspection
Attention	vllm-flash-attn (FA3) fork, custom build `_vllm_fa3_C.d9.so`
GEMM	DeepGEMM (FP8 ue8m0, extended K), Humming W4A16, FlashInfer GDN kernels
Compiler stack	torch.compile / Inductor, Triton, CUTLASS 3.x
Profiling	Nsight Compute 2026.2.1, Nsight Systems, `cuobjdump`
Benchmark client	`aiperf` (scenario `inferencex-agentx-mvp`)
Evaluation	`lm-eval` (GPQA-Diamond CoT), RULER, internal AST-based QA probe

6.3 Serving Hyperparameters

The template hyperparameter table (learning rate, batch size, epochs, optimiser, scheduler) does not apply: nothing is trained. The parameters that determine our result are serving parameters, listed below.

Hyperparameter	Value	How it was chosen
`max_model_len`	500,000	Mandatory - see Section 3.1
`gpu_memory_utilization`	0.87	Highest value that leaves the scorer OOM-free (Section 5.2)
`block_size`	128	Required by the d9 kernel; changes the attention block from 1,568 to 1,664
`max_num_batched_tokens`	16,384	Chunk size trading head-of-line blocking against prefill efficiency
`num_speculative_tokens` (K)	7	DFlash2 trained block size is 8; K > 7 needs two drafter passes
n-gram `N` / window	6 / 4,096	N < 6 produced false matches; window bounded by kernel cost
ASD (`B, g, M`)	(16, 0.25, 2)	Swept; B=8 is the paper value and within noise (Section 4.5.5)
Humming scheme / group	int4 / 64	Group 64 keeps quantisation error at the noise level
Humming `MAX_TOKENS`	32 (shipped)	Extension to 64 measured but held back (Section 8.1, E10)
Proxy `KEEP`	**0.20**	Knee of the quality/latency curve - Section 9.3
Proxy `MIN` / `HEAD` / `TAIL`	4,096 / 256 / 512	Protects short messages and message boundaries
CPU KV offload	143-150 GiB	Auto-sized to available host RAM at boot
Random seed	20260707	Fixed by the organiser

6.4 Benchmark Protocol and Measurement Noise

Every configuration in this report is measured with **one unmodified command** - the organiser own. Only the server configuration changes between runs.

```
aiperf profile --scenario inferencex-agentx-mvp \
  --public-dataset semianalysis_cc_traces_weka_062126 \
  --model Qwen/Qwen3.8-27B-FP8 --tokenizer /mnt/models/hf/Qwen3.8-27B-FP8 \
  --tokenizer-trust-remote-code --endpoint-type chat \
  --max-context-length 500000 --concurrency 20 \
  --benchmark-duration 900 --random-seed 20260707 --workers-max 4
```

The noise floor is 0.0204 ERS, and this governs how every result in this report may be read. We established it by discovering that two archived runs we had treated as different were in fact *identical configurations* - same ASD parameters, same n-gram parameters, prune keep 50.0331% versus 50.0349%, same command, same seed, same host, same day - and scored **0.7857 and 0.8061**. The previously quoted figure of "about 0.006" came from the round-2 benchmark (concurrency 11, max context 204,800) and does not survive the move to concurrency 20 at 500k, where the number of requests completed in 900 s varies much more.

We calibrated a second, independent noise scale for speculative decoding. The d9 kernel is provably **bitwise identical** to its predecessor at the kernel level (confirmed on 900 of 900 measured cases, Table 10b), so the draft acceptance rate *must* be unchanged between two configurations that differ only by it. Measured tokens per step: **2.71 versus 2.87**, giving a noise band of **+/-0.16 tokens per step (about 6%)**.

Consequently we adopt three rules, applied throughout Sections 7-9: (i) any ERS delta below 0.0204 from a single pair of runs is reported as **not distinguishable from noise**, never as an improvement; (ii) such steps are judged instead on **low-noise physical indicators** - prefill ms/request, TTFT p50, TPOT p50, tokens per step; and (iii) where a delta matters we use **per-request paired comparison** (matching requests by trace, turn and kind across two runs) rather than the difference of aggregates.

Configuration verification. Because a mis-started server can waste a 25-minute benchmark, every run reads back both `docker logs` and `docker inspect` to confirm the server is running the configuration it claims - for example `[kpatch] d9: BAT` plus `[d9_kvcache] installed ... V^T pre-permuted (colmajor)` for the attention step, and `speculative_config=SpeculativeConfig(method=dflash, num_spec_tokens=7)` for speculative decoding. Speculative-prefill runs additionally count containers and call `/prune_stats` *before* the benchmark starts: runs without it return `1 container | HTTP 404` and runs with it return `2 containers | HTTP 200`.

7. EXPERIMENTAL RESULTS

7.1 Main Results

Method	ERS (official BTC scale)	Requests / 900 s	TTFT p50 / p90 (ms)	TPOT p50 / p90 (ms)	Dead TTFT / TPOT
Baseline (stock vLLM v0.28)	0.2327	138	927 / 4,637	31.01 / 53.47	35 / 24
Ours - intermediate (KEEP 0.5)	0.8103	474	245 / 1,080	3.77 / 5.95	13 / 0
Ours - KEEP 0.3	0.8404	487	221 / 710	3.71 / 5.92	10 / 1
Ours - KEEP 0.2 (v2.3)	0.8544 / 0.8545	494 / 496	210 / 564	3.58 / 5.43	4 / 0
Ours - final (+ scorer v3)	**0.8586 / 0.8592**	**491 / 501**	**204 / 542**	**3.56 / 5.50**	**4 / 0**

Table 18. Main results, all measured with the identical organiser command. Two figures indicate two independent runs of the same configuration. Zero request errors and zero preemptions throughout.

The final configuration is verified **from the packaged directory**, not from a live lab server: `chain_verify.sh` stops the lab containers, starts the packaged stack, runs the 900-second benchmark and pairs the result against the reference run. Two such end-to-end verifications produced **0.8586 and 0.8592**, both with 4 dead-TTFT and 0 dead-TPOT requests.

Server-side state at the final configuration: prompt **42,059 tokens per request** after pruning, **2,450 KV tokens computed (5.8%)**, **prefix-cache hit 88.9%**, queue **2.2 ms**, prefill **219 ms**, speculative acceptance **54.8%** at **4.83 tokens per step**, n-gram override **32.1%**, **preemption 0**. Proxy side: 176 messages cut at 20.0% retention, one BM25 refill, **80 ms per request**.

7.2 Improvement over Baseline

Dimension	Baseline	Ours	Absolute	Relative
ERS	0.2327	**0.8586**	**+0.6259**	**+269% (3.69x)**
s_ttft	0.3062	0.8336	+0.5274	+172%
s_tpot	0.1592	0.8835	+0.7243	+455%
Requests served in 900 s	138	491	+353	+256%

TTFT p50	927 ms	204 ms	-723 ms	-78%
TTFT p90	4,637 ms	542 ms	-4,095 ms	-88%
TPOT p50	31.01 ms	3.56 ms	-27.45 ms	-88% (8.7x)
Tokens per decode step	1.00	4.83	+3.83	4.8x
Prefill per request	3,589 ms	219 ms	-3,370 ms	-94%
Queue per request	815.7 ms	2.2 ms	-813.5 ms	-99.7%
Requests dead (TTFT / TPOT)	35 / 24	4 / 0	-31 / -24	-89% / -100%

Table 19. Improvement over the organiser baseline, same command, same host class.

Why the improvement is this large. The three lines of work attack three different terms and none of them substitutes for another. Speculative decoding plus the n-gram overlay multiplies tokens per forward step by **4.8x**, which is where the 8.7x TPOT improvement comes from. Speculative prefill cuts the prompt by 62%, which is most of the 94% prefill reduction. And the kernel work plus the KV-capacity work removes the queueing spiral - the baseline held 6.97 requests resident with 816 ms of queueing, whereas the final system holds 2.06 with 2.2 ms - which is why the *tails* (TTFT p90 -88%, dead requests -89%) improve even more than the medians. The improvements compound: shorter prompts mean fewer resident KV pages, which means faster decode steps, which means shorter residency, which means a shorter queue.

8. EXPERIMENT HISTORY

8.1 Development Trajectory

ID	Experiment / change	ERS	Key observation
E0	Baseline: stock vLLM v0.28, prefix caching, FP8 KV, no speculation	0.2327	Decode-bound: 76% of requests in the 20-50 ms TPOT band; near saturation at 6.97 concurrent
E1a	**Disable FP8 two-level accumulation** (`fa-lazy0`), measured alone on the round-2 scale	0.7015 to **0.7677**	**Macro-only A/B against a same-toolchain control, paired t = 11.12 (+0.0574 vs the shipped binary on that scale).** Kernel 2.0-2.5x at every shape. The largest single win of the competition, and the only change that raised s_ttft **and** s_tpot together. Carries a correctness debt (Section 4.4.3)
E1b	+ full FA3 attention package (E1a + evo3-evo8 evolution + d9 V pre-permuted in cache)	0.4096	**+17.69 points** on the competition scale. Far above the kernel speedup alone - it also unwinds the queueing spiral (queue -81%, prefill -53%, 138 to 231 requests). E1a is the dominant component; the d9 layout change alone measures +0.0039 (below noise)
E2	+ GEMM Humming W4A16 (int4, group 64)	0.4135	+0.39, **below the noise floor**. Physical indicators consistent (TPOT p90 -14.3%) but it also costs 9.4% of the KV cache
E3	+ DFlash2 speculative decoding, K = 7	0.3296	**-8.39 points - a regression.** Decode improves exactly as promised; TTFT collapses because the drafter evicts 28.7% of the KV cache (Section 4.5.2)
E4	+ scheduler fixes (nodrop, emit) and CPU KV offload 120 GB	0.4028	+7.32 from E3. Recovers the regression to roughly break-even, no more
E5	+ **Speculative Prefill** (proxy, KEEP 0.5)	0.5628	**+23.32 from E3** - three times the combined effect of E4. Reducing demand beats adding capacity
E6	E5 + all of E4 together	0.6511	The two paths are additive (7.32 + 23.32 = 30.64 predicted vs 32.15 measured)
E7	+ full production stack: ptail v2, gdncg v3.2, fastquant, fusemlp, K2, Rust frontend, lm_head W4, prefix-hit tuning	0.8103	Accumulated systems work; TTFT p50 245 ms, 474 requests, 0 errors
E8	+ **ASD** (B=16, g=0.25, M=2) + **n-gram drafter** (N=6, window 4096)	0.8061*	Acceptance 39.6% to 56.2%, 4.90 tokens/step. *Different host; the same configuration measures 0.8103-0.8368 across hosts - see the host-variance note in 6.1
E9	Proxy KEEP 0.5 to 0.3, then to 0.2 with the v2.3 query scorer and BM25 refill	0.8545	+0.014 per KEEP step, paid for in long-context accuracy (Section 9.3)
E10	Humming M <= 64 with a per-M tile table (lab)	0.8507 vs 0.8542	**Held back.** Paired TPOT comparison shows the predicted +0.002, but a single benchmark cannot resolve it against

			+/-0.006 noise. Not shipped without more runs
Final	**E9 + lossless scorer v3 (async, no-pack, trimmed layer 15)**	**0.8586 / 0.8592**	Scorer 255 to 204 ms per cut message, byte-identical output on 17/17 cases. Two verification runs

*Table 20. Development trajectory. **E1a is on the round-2 scale** (concurrency 11, floors at 0) because that was the scoring formula in force when it was measured, and is not directly comparable to the other rows; every other row is on the competition scale. E0 and E1b-E6 come from the controlled ablation ladder of Section 9, measured on one host with a minimal patch set; E7 onward are production configurations.*

8.2 Important Negative Results

We report these because each cost real time and each carries a transferable lesson.

32. **Speculative decoding as a standalone optimisation (-8.39 points).** Expected to be the single largest win, based on published 2x throughput figures. It is a net loss until prompt size is reduced, because the drafter competes with the prefix cache for GPU memory. **Lesson:** on memory-bound long-context serving, evaluate any component by the KV cache it displaces, not by its FLOPs.
33. **Relaxed-threshold acceptance appeared free and was destroying quality.** GPQA showed 0.707-0.712, unchanged. The measurement was greedy, and the greedy branch of the rejection kernel never reaches the relaxed rule. Under the sampling configuration the workload actually uses, quality falls to **0.591-0.596**. **Lesson:** evaluate the acceptance rule under the exact decoding parameters used in production; a quality gate that does not exercise the changed code path is worse than no gate.
34. **`nodrop` did not do what its documentation said.** It was described as reducing recomputed KV by 700-1,900 tokens per request. Measured, recomputed KV *increased* (62,286 to 65,693) and the prefix-hit rate did not recover (43.3% to 38.7%). Its entire +3.65 points came from **decode** (TPOT p50 14.58 to 11.26 ms). The arithmetic explains it: it can rescue at most one block (1,664 tokens) while the configuration was losing 43,414 tokens per request. **Lesson:** attribute a gain to the mechanism the metrics support, not to the mechanism in the docstring.
35. **`emit` works, and buys nothing.** It reduced TTFT p50 by 76% (5,086 to 1,222 ms) - and the s_ttft gain (+0.029) was almost exactly cancelled by an s_tpot loss (-0.023), net +0.003. The rescued TTFT was still above the 2,000 ms ceiling, so it earned no points. **Lesson:** under a saturating metric, a large latency improvement in the wrong band is worth nothing.
36. **W4A8 was slower than W4A16.** Halving activation traffic should have relieved the shared-memory bottleneck; measured 69.6 us versus 47 us at M = 64 across 86 swept configurations. Rejected without needing to evaluate its numerics.
37. **Four hand-written Humming kernel variants changed nothing.** Deeper `wgmma` pipelining, a larger producer register budget, removing the producer `fence.proxy.async`, and their combination were all bitwise identical and all within noise. **Lesson:** when Nsight Compute says a pipe is at 49-58% and the operand layout forces `bm/16` re-reads, the ceiling is structural - stop rewriting the schedule.
38. **Extending CUDA graphs to 6,144 tokens ran out of memory.** Full-graph capture needed 4.6 GB with 4.2 GB free and killed the server. The expected gain (warm TTFT -40 to -50 ms) was real but unreachable without giving up KV cache.
39. **Longer n-gram drafts (K 7 to 15) were analysed and rejected.** The post-EOS region is 53% of steps with 31% match rate at 94% acceptance, suggesting 4.9 to about 6.2 tokens per step. But DFlash2 has block size 8 (K > 7 needs a second drafter pass, +1 ms), the fused GDN path caps at 8 MTP tokens, and verifying 16 tokens per request costs +0.6 ms - cost approximately equals benefit.
40. **Adaptive KEEP by message length was implemented and dropped.** At equal token budget it gained 4-5 RULER points but did not move ERS, so it added a lossy mechanism for no scored benefit.
41. **A latent crash in `_ptail`.** Without the pruning proxy, `_ptail` trips `assert blk.block_hash_num_tokens is not None`, kills the EngineCore after about 6 minutes of load and 500s every subsequent request. The shipped configuration does not hit it because pruning keeps contexts below the boundary - but that is **data, not design**. We recommend and document a guard that falls back to the original path instead of asserting. We report this because it is exactly the class of defect that a competition score cannot see and a production deployment would find at 3 a.m.

9. ABLATION STUDY

9.1 Controlled Ablation Ladder

To attribute the score to individual techniques we built a **minimal patch stack** rather than ablating the production build, because the production patches assume one another. Five modules only (`sitecustomize`, Humming, d9 KV cache, scheduler fix, offload fix) sit on top of the **unmodified organiser baseline command**, with mounts and environment variables added one step at a time. Every configuration is measured with the identical benchmark command; the score is 100 x ERS.

#	Configuration	ERS	Score	Delta
K0	Baseline (organiser)	0.2327	23.27	-
K2	+ FA3 attention kernel (d9)	0.4096	**40.96**	**+17.69**
K4	+ GEMM Humming W4A16	0.4135	41.35	+0.39 *
K5	+ DFlash2 K = 7	0.3296	32.96	**-8.39**
K6	+ nodrop	0.3661	36.61	+3.65
K7	+ emit	0.3693	36.93	+0.32 *
K8	+ CPU KV offload 120 GB	0.4028	40.28	+3.35
K9	K5 + speculative prefill (KEEP 0.5)	0.5628	**56.28**	**+23.32** vs K5
K10	K9 + nodrop + emit + offload	0.6511	**65.11**	**+8.83** vs K9

*Table 21. Controlled ablation. * marks a delta below the 0.0204 ERS (2.04 point) noise floor, which therefore cannot be concluded from ERS alone.*

```
baseline                                  23.27
  + attention (FA3 d9)                    40.96   +17.69
    + Humming                             41.35   +0.39  *
      + DFlash2                           32.96   -8.39  <- speculative decoding LOSES
        |- + nodrop + emit + offload       40.28   +7.32
        |- + speculative prefill           56.28   +23.32
        \- + speculative prefill + all     65.11   +32.15
```

Figure 2. Ablation tree. The branch structure is the finding: the same three infrastructure techniques are worth 7.32 points, while speculative prefill alone is worth 23.32.

9.2 Contribution of Each Component

What the +17.69 of step K2 actually contains. The `d9` binary benchmarked at K2 is the whole attention package: two-level accumulation disabled, the `evo3-evo8` schedule evolution, and the V-colmajor cache layout. Only the last of these could be toggled at run time, and on its own it measures **+0.0039 - below the noise floor**. The dominant component is the **two-level disable**, measured separately in a macro-only A/B during development (Table 20, E1a). We state this explicitly rather than letting the ladder imply that a cache-layout change is worth 17.69 points. The three-tier kernel measurement of Section 4.4.5 (Tables 10d-10e) gives the same decomposition at the kernel level: of the 3.4-3.6x attention speedup over stock vLLM at 128k, 2.6-3.2x is the two-level disable and 1.07-1.26x is d9's own, bitwise-neutral work.

A - FA3 attention kernel: +17.69 points, the largest single contribution. The baseline sits near collapse: 6.97 concurrent requests, 815.7 ms of queueing and 3,589 ms of prefill per request. Faster attention lets requests exit sooner, so fewer accumulate (6.97 to 5.44), so each decode step reads less KV, so they exit sooner still. The spiral unwinds: **queue -81%, prefill -53%, requests served 138 to 231, requests dead on TPOT 24 to 7.** At the kernel level the same change measures only "attention prefill -10 to -14%" - the +17.69 is mostly the de-saturation effect, and we say so rather than claiming a 17-point kernel.

B - Humming W4A16: +0.39 points, below the noise floor. The two runs are nearly indistinguishable (both 231 requests, both 5.44 concurrent, prefill 1,680 versus 1,683 ms); only TPOT p90 moves consistently (30.76 to 29.17 ms). The physical explanation is that Humming reduces MLP **weight** bandwidth, but on this baseline decode is bound by **KV** traffic (5.44 requests x about 108k tokens), so the term it optimises is a small share of the step. It also **costs 9.4% of the KV cache**. We ship it on the

evidence of the low-noise indicators (TPOT p50 -6.1%, p90 -14.3%, prefill unchanged), not on the ERS delta.

C - Speculative decoding: -8.39 alone, +23.76 in combination. Fully analysed in Section 4.5.2. The important ablation result is the **interaction**, not the main effect.

Interactions. Two matter. First, speculative decoding and speculative prefill are **strongly complementary**: DFlash2 is -8.39 alone but +14.93 with pruning and +23.76 with pruning and infrastructure, because pruning returns the KV cache that the drafter took. Second, speculative prefill and CPU KV offload are **additive but mechanistically different** - offload *adds capacity* at the cost of PCIe bandwidth, pruning *reduces demand*; predicted 30.64 versus measured 32.15 combined.

An ablation that produced a negative result about our own kernel. In the *production* stack we could also ablate d9 against its predecessor, and the result was instructive: **+0.0039 ERS, below noise, with mutually contradictory indicators** (prefill 329 to 344 ms worse, TTFT p90 777 to 900 ms worse, TTFT p50 237 to 219 ms better, TPOT p50 6.65 to 6.37 ms better). The reason is that by then the stack had a **84.58% prefix-cache hit with only 5.3% of the prompt recomputed** - on a 65,936-token prompt the server truly prefills about 3,500 tokens, and total prefill is 329 ms. d9 optimises *cold* prefill attention; at that point there was almost nothing left for it to optimise. The kernel improvement is real and was verified in isolation, but **its benefit does not transfer to this scoreboard once prefix caching is working well.** This is exactly the kind of conclusion that only appears when ablating with the organiser command instead of a kernel microbenchmark.

9.3 Quality/Latency Ablation of Speculative Prefill

KEEP is the one parameter that trades score directly against output quality, so we swept it and measured all four axes on the same stack.

KEEP	ERS (BTC)	RULER 32k	RULER 128k	Internal QA, temp 0	Internal QA, temp 1 (5 runs)
1.0 (no pruning)	TBD	TBD	TBD	**97.6%**	97.1% +/- 2.0
0.5	0.8103	82.4	71.2	90.2%	82.9% +/- 4.9
0.3	0.8404	81.1	72.3	73.2%	66.8% +/- 2.8
0.2 (shipped)	**0.8545**	71.0	67.0	70.7%	72.7% +/- 3.2
0.15	0.8614	68.0	64.1	70.7%	62.9% +/- 7.6

Table 22. Quality/latency trade-off. Internal QA = 41 AST-derived questions over 14 real source files delivered as tool results, scored automatically against ground truth.

ACTION REQUIRED - missing control row. *The KEEP = 1.0 row (pruning fully disabled) has no ERS and no RULER numbers, because the proxy was never benchmarked in pass-through mode on the BTC scale and RULER was only ever run through the proxy. This is the **control condition** for the whole table: without it we can state the relative cost of each KEEP step but not the absolute quality and latency cost of using speculative prefill at all, and a reviewer can reasonably ask exactly that. Three runs are needed, about 30 GPU-minutes each plus RULER time, on one host in one session: **(i)** ERS on the BTC command with `KEEP=1` (or the proxy bypassed) - note that this may make long requests *worse*, which is itself the result; **(ii)** RULER 32k and **(iii)** RULER 128k against the engine directly on port 8001, bypassing the proxy. Delete this note and unhighlight the row once the three numbers are filled in. If the runs cannot be done before submission, replace the TBD cells with "not measured" and add one sentence to Section 14 (Limitations) stating that the no-pruning control was not measured on the final scale.*

The curve is steep on both axes: each KEEP step is worth roughly +0.014 ERS and costs several points of long-context accuracy. **We selected KEEP = 0.20 as the knee**: below it, ERS gains flatten (0.15 adds +0.007) while quality variance explodes (standard deviation 3.2 to 7.6). We note explicitly that this is a *deliberate, quantified* trade-off. ERS does not measure it, so without this table the choice would have been invisible.

The failure structure is informative. The first question in each conversation - the one whose intent the attention scorer can see while pruning - is answered correctly 91-94% of the time even at KEEP 0.2-0.3. The loss concentrates in the **second and third turns**, which ask about material discarded for the first question (0.5: 79%/75%; 0.3: 47%/62%). The most fragile category is counting function parameters (40% at KEEP 0.3). Encouragingly, the model **does not hallucinate**: it answers "I do not see the string literal ... in the provided file content", i.e. it knows the content is missing.

10. FAILURE ANALYSIS

The template failure-analysis section assumes a prediction task with qualitative error cases. For an inference-optimisation task the analogous question is: ***where is score still being lost, and why?*** *We answer that below, and add a second subsection on quality failures, since several of our techniques are lossy.*

10.1 Where Score Is Still Lost

At the final configuration the residual loss decomposes as follows (measured, not estimated):

Band	Requests	ERS lost	Interpretation
TTFT 100-250 ms	211	0.0167	**Largest single bucket.** Just above the 100 ms floor; almost entirely proxy overhead
TTFT 250-500 ms	127	0.0300	**Largest loss.** Warm-turn prefill plus proxy
TTFT 500-1,000 ms	40	0.0213	Head-of-line blocking behind long prefills
TTFT 1-2 s	16	0.0135	Same cause, worse instance
TTFT > 2 s (dead)	4	0.0030	Unrecoverable; rescuing them to the ceiling is worth 0.0000
TPOT 2-5 ms	401	0.0400	The steady-state decode step (12.8 ms wall for about 4.8 tokens)
TPOT 5-10 ms	74	0.0147	Steps with 5-8 concurrent requests
TPOT > 10 ms	11	0.0063	Long prefill interleaved into a decode step

Table 23. Residual score loss. Total: 0.0845 from TTFT and 0.0613 from TPOT.

The decisive observation is that **TTFT p50 does not grow with input length**. Requests with ISL < 10k cost about 240 ms, the same as requests with ISL > 200k. This is not a long-context problem; it is a **fixed per-request overhead of about 200 ms**, sitting directly on top of a 100 ms scoring floor. Its composition at the shipped configuration is: proxy scoring about 80-98 ms, HTTP and queueing about 20 ms, and warm prefill of two steps about 90-130 ms (about 2,450 genuinely new tokens, roughly 84 ms of GPU kernel time). This is why our final optimisation effort went into making the **scorer faster** rather than making the model faster - and why the remaining headroom is there too.

10.2 Failure Categories

Failure type	Frequency	Example / signature	Root cause
Dead on TTFT (> 2,000 ms)	4 / 494	Long prefill arriving while another long prefill occupies the chunk budget	Head-of-line blocking; vLLM 0.28 lacks `max_num_partial_prefills`
Dead on TPOT (> 50 ms)	0 / 494	-	Eliminated by speculative decoding plus n-gram overlay
Score clamped away by the floor	96 / 494 (not a loss)	TTFT < 100 ms already scores 1.0	Further TTFT work cannot pay on these requests
Long-context content loss	KEEP-dependent (Table 22)	Turn 3 asks about a function pruned when answering turn 1	Speculative prefill is lossy by design; frozen per message
Silent numerical loss at 128k+	Not observed in scored metrics; risk quantified	FP8 fast accumulation swamps small attention terms behind an early sink	FP8 tensor-core accumulator is not true FP32 (Section 4.4.2)
Acceptance-rule quality loss	Detected and removed before submission	Relaxed tau: reasoning never terminates in 32k (57-58 of 198 empty)	Non-distribution-preserving acceptance; masked by greedy evaluation
Latent engine crash	0 in shipped configuration	`_ptail` assertion kills EngineCore after about 6 minutes without the proxy	Boundary condition on the last prompt block hash; guarded by data, not by design

Table 24. Failure categories. Rows 4-7 are quality or robustness failures that the competition metric cannot observe; we list them because we consider them real.

10.3 Root Cause Analysis

Mapping the residual failures onto their origin:

- **Scheduling / resource limits (dominant).** The remaining TTFT tail is head-of-line blocking: a 16k chunk of a very long prefill delays a short request behind it. The natural fixes - reducing the chunk to

8k and scheduling shortest-prefill-first - are not directly available in vLLM 0.28, which exposes only `long_prefill_token_threshold` and FCFS/priority policies. We estimate +0.005 to +0.015 from a ported shortest-job-first scheduler and measured +0.006 for such a patch under the round-2 scale.

- **Fixed pipeline overhead (dominant).** The 200 ms floor described above. Roughly half is the proxy scorer, which is why we optimised it (255 to 204 ms per cut message, byte-identical output) rather than the model.
- **Optimisation, not model.** Nothing in the residual loss is attributable to the model or its weights. The decode step is 12.8 ms wall with the GPU idle only 0.84 ms (7%), issuing 1,350 kernels of which 999 are under 10 microseconds - so about 3-4 ms per step is small-kernel launch tax rather than useful work.
- **Domain shift / data dependency.** The one genuine data dependency is that our KEEP setting was chosen on this trace distribution. A workload whose long messages are information-dense rather than repetitive (the SpecPrefill authors observe the same on few-shot benchmarks) would need a higher KEEP.
- **Not noise.** We ruled noise out explicitly for every conclusion above by requiring either a delta above 0.0204 ERS or agreement among low-noise physical indicators (Section 6.4).

11. COMPUTATIONAL EFFICIENCY

Metric	Result
Target model parameters	27B (Qwen3.8-27B-FP8); 64 layers, 16 full-attention + 48 gated-delta-net, head_dim 256, GQA 24/4
Auxiliary models	DFlash2 drafter (about 3.59 GB bf16); Qwen2.5-0.5B-Instruct scorer truncated to 16 layers (about 1.5 GB)
Model size on GPU	about 27 GB FP8; MLP weights re-quantised to int4 group 64 at load time
Training time	**None** - no training or fine-tuning was performed
TTFT (p50 / p90 / p99)	**204 / 542 / 1,739 ms**
TPOT (p50 / p90)	**3.56 / 5.50 ms**
Throughput	**491 requests / 900 s** at concurrency 20; about 1,111 output tokens per request; prefill about 10,894 tokens/s
Tokens per forward step	**4.83** (acceptance 54.8%)
Peak GPU memory	about 125 GB engine (utilisation 0.87) + about 2.5 GB scorer; KV cache about 1.82 M tokens on GPU
CPU memory	143-150 GiB used as second-level KV store
Hardware	1x NVIDIA H200 (143 GB), 16 vCPU
Cold start	about 9-10 minutes (weight load, torch.compile, CUDA-graph capture)

Table 25. Computational efficiency of the final system.

Trade-offs we accepted, stated explicitly.

- **Quality for latency.** Speculative prefill discards 80% of long-message tokens. Internal long-context QA falls from 97.6% to 70.7% at KEEP 0.2 (Table 22). GPQA is unaffected because its prompts never reach the pruning threshold - which is a property of GPQA, not evidence of safety.
- **Memory for latency, in both directions.** Humming spends 9.4% of the KV cache to reduce weight bandwidth; CPU KV offload spends PCIe bandwidth to buy 143 GiB of capacity; the drafter spends 28.7% of the KV cache to raise tokens per step. Every one of these is a memory-for-latency trade, and on this workload the KV cache is the currency.
- **Numerical precision for compute.** FP8 KV cache halves KV footprint; disabling two-level accumulation avoids a roughly 2x slower prefill attention path at the cost of accumulation error at very long context (Sections 4.4.2-4.4.3).
- **Complexity for score.** The system is two containers with about fifteen runtime patches. Section 15 discusses what this means for production.

12. ROBUSTNESS AND GENERALIZATION

Different parameter settings. The KEEP sweep (Table 22) shows monotone, well-behaved response from 0.5 to 0.15. ASD was swept over B in {0, 8, 16} and g in {0.25, 0.5}; the B-to-ERS curve is flat beyond B = 8, matching the plateau reported in the ASD paper, so the setting is not on a knife edge. The Humming tile table was validated across M in {1, 8, 16, 24, 32, 40, 48, 56, 64, 80, 96, 128}.

Different hardware. All results are on H200 (sm90a). Two components are hardware-specific by construction: the d9 attention kernel targets Hopper TMA and `wgmma` layouts, and the Humming kernel targets `wgmma.m64nMk16`. On Blackwell both would need re-derivation, though the *analysis* - that the producer transpose is on the critical path, and that W4 GEMM is shared-memory-bound at moderate M - should transfer. The systems and algorithmic contributions are hardware-independent.

Different hosts. We measured genuine host-to-host variance: the same configuration scored 0.9206 and 0.9168 on two hosts (round-2 scale), and 0.8103 versus 0.8368 across hosts at KEEP 0.3-0.5. Every comparison in this report therefore uses a same-host reference, and we flag the one table row (E8) where that was not possible.

Noisy and unseen inputs. The proxy is guarded: messages under 4,096 tokens are never pruned; messages with fewer than 4 natural units are never pruned; HEAD 256 and TAIL 512 tokens are always retained. If the scorer fails (for example CUDA OOM) it falls back to a compression-based heuristic rather than failing the request - we measured 0 fallbacks at the shipped memory setting and 13 of 88 at the more aggressive one, which is why we ship the conservative one. Across 900-second runs at the final configuration we observed **0 request errors and 0 preemptions**.

Limits of our generalisation evidence. We did not test a different model family, a different GPU, or a non-agentic workload. Our strongest technique (speculative prefill) exploits the specific property that long tool outputs are information-sparse and re-sent repeatedly; on an information-dense workload we would expect it to require a much higher KEEP - the SpecPrefill authors report exactly this failure mode for few-shot prompts.

13. DISCUSSION

13.1 Why the Method Works

The single sentence that explains our result is: **on a long-context agentic workload the binding resource is KV-cache capacity, and the highest-leverage optimisations are the ones that reduce demand for it.** Every mechanism in Section 4 is a variation on that theme. Speculative prefill cuts prompts by 62% and is worth 23 points. CPU KV offload adds capacity and is worth 3. Speculative decoding *consumes* capacity and is worth -8 until the demand side is fixed, after which it is worth +24. Even the attention kernel is worth three times its microbenchmark speedup, because faster requests are resident for less time and resident requests are what fill the cache.

13.2 Which Contribution Matters Most

Two answers, and they point at the same kernel.

By ERS, the largest single step from the baseline is the attention package: +17.69 points on the competition scale (0.2327 to 0.4096, Table 6), whose dominant component is removing FP8 two-level accumulation - the only change in the entire competition that raised s_ttft and s_tpot together. After it, **speculative prefill** (+23.32 points from a common base, versus +7.32 for all our infrastructure work combined) is the largest algorithmic contribution.

By engineering value we would rank the accuracy work alongside it, even though it contributes no ERS. The two are inseparable: the speedup can be deployed responsibly only because the debt it creates was found, measured and repaid in a build that can replace the submitted kernel wherever long-context accuracy must be guaranteed (the submitted configuration itself serves the unfixed d9 kernel, Section 6.2). A deployment that took the speed result from Section 4.4.2 without Sections 4.4.3-4.4.4 would be fast, would score well, and would be discarding up to 90% of its attention result at long context with no metric able to tell. Our flush recovers about 96% of that error for an 11.6% cost. It is also the only

contribution whose value does not decay as the rest of the stack improves - notably, the d9 layout work *did* decay to below the noise floor once prefix caching reached 84.6%.

Beyond that, the honest answer is that the ranking depends on where you start. The attention kernel is worth 17.69 points on a saturated baseline and, by the time the stack reaches 84% prefix-cache hit, its measured contribution falls below the noise floor - because there is barely any cold prefill left to accelerate. Speculative prefill, conversely, becomes *more* valuable as the rest of the stack improves, because it is the only technique that reduces both the prefill and the decode working set at once. Reporting a single ordering without the context would be misleading, so we report both.

13.3 What Surprised Us

42. **A one-line build flag was worth more than every algorithm we implemented.** Upstream data said disabling two-level accumulation would buy 5-13% and cost accuracy - a plainly bad trade, and we nearly did not test it. At head_dim 256 it is worth 2.0-2.5x at the kernel and carries the +17.69-point attention step on the competition scale, where speculative decoding on its own is a loss of 8.39 points. The lesson is not "always disable it" but that **published kernel trade-offs are measured at one head_dim and do not transfer**; the cost of testing the assumption was one rebuild.
43. **That same flag makes the kernel numerically broken at long context, and nothing in the standard toolchain reveals it.** We took the speedup and then went looking for what we had paid: up to 90% of the attention result is discarded at 262k, the error is invisible to ERS, invisible to GPQA (whose prompts are a few thousand tokens), and invisible to the very microbenchmark written to detect it - because Gaussian test inputs produce the one softmax shape in which fast accumulation is safe. The existing `rel_l2 = 2.6e-2` numbers looked like proof of correctness and were nothing of the kind.
44. **Speculative decoding was a net loss.** We expected the largest single win and measured -8.39 points. The mechanism - drafter weights evicting the prefix cache - is obvious in hindsight and is invisible in every published benchmark of the technique.
45. **A quality gate that never exercised the code it was gating.** Relaxed acceptance measured "no degradation" for weeks because GPQA runs greedily and the greedy branch skips the relaxed rule. Under the real sampling configuration it cost 12 points.
46. **A kernel code path in a widely used project that could never have executed.** The V-colmajor path in the vLLM FlashAttention fork is blocked by four independent barriers, so every previous measurement of it - including our own team earlier conclusions - was measuring a hidden `contiguous()` copy.
47. **Our measurement noise was 3.4x larger than we believed.** The documented figure of about 0.006 ERS came from the round-2 benchmark. At concurrency 20 and 500k context the true figure is 0.0204, which invalidated several earlier conclusions and forced us to re-derive them from low-noise indicators.
48. **The best kernel work does not always transfer to the score.** d9 is a genuine, bitwise-verified kernel improvement, and once prefix caching reached 84.6% its end-to-end contribution became indistinguishable from noise.

13.4 Generalisable Insights

49. **Evaluate an inference optimisation by the KV cache it displaces, not by its FLOPs.** On long-context workloads with high prefix reuse, GPU memory is the scarce resource and every published speedup measured at fixed KV capacity should be re-derived.
50. **Demand reduction dominates capacity addition.** Cutting prompt tokens improved prefill, decode and queueing simultaneously and was three times more valuable than adding 120 GB of offload capacity.
51. **A quality gate must exercise the code path it gates, under production decoding parameters.** Temperature-0 evaluation of a sampling-branch change is not evidence.
52. **Random-input microbenchmarks are structurally blind to accumulation errors.** Gaussian Q and K give a near-uniform softmax, which is precisely the regime where FP8 fast accumulation is safe. Testing for swamping requires adversarial inputs (a sink plus a long tail), and we contribute such a test.

53. **Establish the noise floor before drawing conclusions.** Running the *same* configuration twice was the single highest-value experiment we performed; it changed how we read every other result.
54. **Profile the binary you serve, under the command you are scored by.** Our two most important negative results (d9 not transferring; `nodrop` not doing what its documentation claimed) are both invisible to microbenchmarks.

14. LIMITATIONS

55. **Output quality is degraded at long context, by design.** Speculative prefill discards 80% of long message tokens; internal long-context QA falls from 97.6% to 70.7% (Table 22). We quantified this and chose the operating point deliberately, but a deployment with strict long-context fidelity requirements would need a higher KEEP and would lose most of the +0.044 ERS that KEEP tuning contributed.

The FP8 accumulation result is quantified but not fully closed. Our flush reduces error at 262k from -90.6% to -3.4% at an 11.6% speed cost, but -3.4% is not zero. The kernel-level tests (Tables 7b, 9, 9b and 10b-10e) and an end-to-end A/B against the flush build on the organiser GPU (Table 9c) have been run; the retrieval and teacher-forced NLL probes did not separate the two kernels, so the downstream cost of the residual error - and of the unfixed kernel - on tasks that need diffuse long-context information remains unmeasured, and RULER was not run. The full upstream fix remains unaffordable at head_dim 256 (measured: 2.62x slower prefill attention and 3.05-3.21x slower decode, Table 10e), and we did not implement the reduced-tile variant that might make it affordable. The submitted configuration serves the unfixed d9 kernel; the flush build is a drop-in replacement where long-context accuracy must be guaranteed.

56. **The no-pruning control was not measured on the final scale.** Table 22 has no ERS and no RULER numbers for KEEP = 1.0, so we can report the relative cost of each pruning step but not the absolute quality and latency cost of using speculative prefill at all. Three runs would close this (Section 9.3).
57. **Some ablation steps could not be isolated.** `K2` and `fusemlp` are hard-wired in the serve script and are constants at every rung of the ladder. Only the V-colmajor/TMA part of the attention work could be ablated - a single `.so` exists, so the +17.69 belongs to the *package* rather than to a single change. The +17.69 also includes an attention block-size change from 1,568 to 1,664 forced by the kernel `assert N % 128 == 0`; we believe this contribution is small (the two KV caches differ by 0.09%) but the control run was cancelled.
58. **The contribution of pruning could not be measured inside the controlled ladder.** With the full patch stack the server does not run reliably without the proxy, for reasons we only partly diagnosed (Section 8.2, item 10 and a second unexplained empty-stream failure). Pruning is therefore *in the background* of the ladder rather than a rung of it. We are explicit that this is a limitation of the measurement session and **not** evidence that pruning is required: the baseline runs without it perfectly well.
59. **Single-hardware, single-model evidence.** All results are on one H200 with one model. Two kernels are Hopper-specific.
60. **Statistical power.** Most configurations were measured once or twice against a 0.0204 noise floor. Steps with small deltas (Humming, d9 in the production stack, `emit`) would need three or more repeats each - about 30 GPU-minutes per run - to be resolved. We report them as unresolved rather than rounding them up.
61. **Operational complexity.** Two containers, about fifteen monkey patches and a custom kernel `.so` are not a maintainable production artefact as-is (Section 15).
62. **One known latent defect.** The `_ptail` assertion (Section 8.2) is currently prevented by data rather than by a guard.

15. REAL-WORLD DEPLOYMENT

If this solution were deployed in a Viettel production environment, the following would have to change.

15.1 Architecture and Scalability

- **The proxy becomes a first-class service, not a sidecar.** Today it is a single aiohttp process holding a 0.5B scorer on the same GPU as the engine. In production it should be a horizontally scalable service with its own small GPUs (or a shared scorer pool), so that scorer latency - currently about half of the 200 ms fixed overhead - can be scaled independently of the engine.
- **The `sha256` pruning cache must be shared and persistent.** It is per-process today. Across replicas it must be a shared store (Redis or similar), otherwise the same message is pruned differently by different replicas and the prefix cache is destroyed - which would remove most of the benefit.
- **Multi-GPU would change the optimal design substantially.** Prefill/decode disaggregation would eliminate the head-of-line blocking that is our largest remaining loss, and tensor parallelism would relax the KV-capacity constraint that shaped every decision in this report. Several of our trade-offs (notably the Humming/KV-cache trade) would need to be re-derived.
- **Routing should be prefix-aware.** With more than one replica, routing a conversation to the replica that already holds its prefix is worth more than any kernel optimisation in this report.

15.2 Reliability, Monitoring and Maintenance

- **Remove the monkey-patch stack.** About fifteen `sitecustomize` patches against vLLM internals will break on the next upgrade. The kernel work (d9, Humming tiles, Triton quantiser) should be upstreamed or maintained as a versioned fork; the policy patches (ptail, scheduler fixes) should become configuration or upstream contributions.
- **Fix the latent `_ptail` assertion** with the fallback guard described in Section 8.2 before any production traffic. It is currently prevented only by the shape of the data.
- **Add a graceful degradation path.** Scorer failure must fall back to no pruning (slower but correct), never to a failed request. Today the fallback is a weaker heuristic.
- **Monitoring must include quality, not just latency.** The competition metric is latency-only, and two of our most dangerous findings (relaxed acceptance, FP8 accumulation) were quality failures invisible to it. Production monitoring needs periodic long-context canaries (RULER-style retrieval at 32k and 128k) alongside TTFT/TPOT dashboards, plus prefix-cache hit rate and speculative acceptance as leading indicators.
- **Re-gate quality after every quantisation change.** Humming, W4 projections and int8 lm_head all alter numerics; each needs a GPQA and RULER run at production sampling parameters, not greedy.

15.3 Cost, Latency and Security

- **Cost.** The direct result is a 3.6x increase in requests served per GPU-second, i.e. roughly a 3.6x reduction in serving cost per request at equal latency targets - before any multi-GPU scaling.
- **Latency SLOs.** We would publish TTFT p95 and TPOT p95 rather than the ERS aggregate, since ERS saturation hides both the very fast and the very slow tails.
- **Security and privacy.** The proxy reads and rewrites full user message content and caches pruned text keyed by content hash. In production that cache is a data store containing user content and needs the same retention, encryption and tenancy isolation controls as the conversation store itself. Content hash keys must be tenant-scoped to prevent cross-tenant cache hits.

16. CONCLUSION

63. **Problem.** Maximise a latency-based Efficiency-Realism Score for Qwen3.8-27B-FP8 served on a single H200 under a long-context agentic workload (prompts averaging 110k tokens, peaks to 996k).
64. **Proposed solution.** A two-stage pipeline: a speculative-prefill proxy that removes 80% of long-message tokens while preserving prefix-cache hits, in front of a vLLM v0.28 engine carrying a custom FP8 attention kernel, a tuned W4A16 MLP GEMM, a Triton FP8 quantiser, and a three-part draft-and-verify chain (DFlash2 + n-gram overlay + ASD verification).
65. **Main contributions.** First, the finding that **removing FP8 two-level accumulation is worth 2.0-2.5x at head_dim 256 rather than the 5-13% the published data predicts** - the dominant component of the +17.69-point attention step on the competition scale (ERS 0.2327 to 0.4096, Table 6), the largest

single step from the baseline and the only one to improve TTFT and TPOT together. Second, the diagnosis and practical repair of the **silent numerical failure that speedup incurs**: up to 90% of the attention result lost at 262k, the correct upstream alternative 2.5x slower than the unfixed kernel, and our periodic scratch-global-memory flush recovering accuracy to -3.4% for an 11.6% cost, together with an adversarial test that exposes a failure mode no existing benchmark reaches. Beyond that: a diagnosed and fixed FP8 attention *speed* bottleneck (58% to 85% of the compute ceiling in ablation) including repairs to four defects that made the relevant vLLM code path unreachable; a W4A16 GEMM tuned across the speculative-decoding M-range with a shared-memory-bandwidth ceiling analysis; the demonstration that speculative decoding is a net loss without prompt reduction; and a correct sampling-branch adaptation of Approximate Speculative Decoding.

66. **Main results. ERS 0.8586-0.8592 versus a 0.2327 baseline - a 3.69x improvement** - serving 491 requests per 900 s instead of 138, with TTFT p50 of 204 ms (from 927 ms), TPOT p50 of 3.56 ms (from 31.01 ms), zero errors and zero preemptions.
67. **Key technical insight.** On long-context serving with high prefix reuse, **KV-cache capacity is the binding resource, and reducing demand for it beats adding it**. This single principle explains the entire ablation: why speculative decoding is a loss alone and a large gain in combination, why prompt pruning is worth three times all our infrastructure work, and why an attention kernel is worth three times its microbenchmark speedup on a saturated system and nothing at all on an unsaturated one.
68. **Future work.** Shortest-job-first prefill scheduling with an 8k chunk to attack the remaining head-of-line loss (estimated +0.005 to +0.015); the "decode diet" - W4 projections, int8 lm_head and W4 drafter, measured at ERS 0.8634 in the lab and pending quality gating (estimated +0.015 to +0.02); closing the FP8 accumulation question with a full RULER 128k A/B; and upstreaming the kernel work.

17. REFERENCES

69. W. Kwon, Z. Li, S. Zhuang, et al., "Efficient Memory Management for Large Language Model Serving with PagedAttention," in Proc. SOSP, 2023. [Online]. Available: https://github.com/vllm-project/vllm
70. L. Zheng, L. Yin, Z. Xie, et al., "SGLang: Efficient Execution of Structured Language Model Programs," in Proc. NeurIPS, 2024.
71. T. Dao, "FlashAttention-2: Faster Attention with Better Parallelism and Work Partitioning," in Proc. ICLR, 2024.
72. J. Shah, G. Bikshandi, Y. Zhang, et al., "FlashAttention-3: Fast and Accurate Attention with Asynchrony and Low-Precision," in Proc. NeurIPS, 2024.
73. vllm-project/flash-attention, PR #104, "Add two-level accumulation for SM90 FP8 FWD to mitigate long-context accuracy drops." [Online]. Available: https://github.com/vllm-project/flash-attention/pull/104
74. E. Frantar, S. Ashkboos, T. Hoefler, and D. Alistarh, "GPTQ: Accurate Post-Training Quantization for Generative Pre-trained Transformers," in Proc. ICLR, 2023.
75. J. Lin, J. Tang, H. Tang, et al., "AWQ: Activation-aware Weight Quantization for LLM Compression and Acceleration," in Proc. MLSys, 2024.
76. DeepSeek-AI, "DeepGEMM: Clean and Efficient FP8 GEMM Kernels with Fine-Grained Scaling." [Online]. Available: https://github.com/deepseek-ai/DeepGEMM
77. Y. Leviathan, M. Kalman, and Y. Matias, "Fast Inference from Transformers via Speculative Decoding," in Proc. ICML, 2023.
78. T. Cai, Y. Li, Z. Geng, et al., "Medusa: Simple LLM Inference Acceleration Framework with Multiple Decoding Heads," in Proc. ICML, 2024.
79. Y. Li, F. Wei, C. Zhang, and H. Zhang, "EAGLE: Speculative Sampling Requires Rethinking Feature Uncertainty," in Proc. ICML, 2024.
80. "Approximate Speculative Decoding," arXiv:2608.03447v3, 2026.
81. J. Liu, J. Chen, and Z. Zhang, "SpecPrefill: Accelerating Time-to-First-Token in Long-Context LLM Inference," arXiv:2502.02789, 2025.
82. Z. Zhang, Y. Sheng, T. Zhou, et al., "H2O: Heavy-Hitter Oracle for Efficient Generative Inference of Large Language Models," in Proc. NeurIPS, 2023.
83. C.-P. Hsieh, S. Sun, S. Kriman, et al., "RULER: What is the Real Context Size of Your Long-Context Language Models?," in Proc. COLM, 2024.

84. D. Rein, B. L. Hou, A. C. Stickland, et al., "GPQA: A Graduate-Level Google-Proof Q&A Benchmark," in Proc. COLM, 2024.
85. vLLM Project, "The State of FP8 KV-Cache and Attention Quantization," blog post, 22 Apr. 2026. [Online]. Available: https://vllm-project.github.io/2026/04/22/fp8-kvcache.html
86. NVIDIA, "CUTLASS: CUDA Templates for Linear Algebra Subroutines." [Online]. Available: https://github.com/NVIDIA/cutlass
87. NVIDIA, "AIPerf: LLM Inference Benchmarking." [Online]. Available: https://github.com/ai-dynamo/aiperf
88. Qwen Team, "Qwen3 Technical Report," 2025. Model: Qwen/Qwen3.8-27B-FP8.

18. EXTERNAL RESOURCE DECLARATION

18.1 External Datasets

Dataset	Source	License	Usage
`semianalysis_cc_traces_weka_062126`	Public dataset shipped with `aiperf`	As distributed with aiperf	The scored benchmark workload (provided by the organiser)
RULER	github.com/NVIDIA/RULER	Apache-2.0	Long-context quality gate at 32k and 128k for speculative prefill
GPQA-Diamond	HuggingFace `Idavidrein/gpqa`	CC BY 4.0 (gated)	Reasoning quality gate for acceptance-rule and quantisation changes
Internal AST-derived QA probe	Generated by us from 14 public vLLM source files	Derived from Apache-2.0 source	Long-context content-fidelity measurement (41 questions); **our own construction**
Shakespeare / synthetic text	Bundled with aiperf	Public domain	Scorer determinism regression tests (17 cases)

18.2 Pretrained Models

Model	Source	Version	Usage
Qwen3.8-27B-FP8	HuggingFace `Qwen/Qwen3.8-27B-FP8`	FP8 checkpoint, unmodified	Target model. **No fine-tuning.** MLP weights are re-quantised to int4 group 64 at load time
DFlash2 drafter	Provided with the competition stack	block size 8, K = 7	Speculative decoding drafter; weights unmodified
Qwen2.5-0.5B-Instruct	HuggingFace `Qwen/Qwen2.5-0.5B-Instruct`	bf16, truncated to 16 layers at runtime	Attention scorer inside the speculative-prefill proxy; **weights unmodified**, only layers 0-15 are executed

18.3 External Code

Repository	Source	License	Components used
vLLM	github.com/vllm-project/vllm	Apache-2.0	v0.28.0 engine: V1 scheduler, PagedAttention, prefix caching, FP8 KV cache, chunked prefill, Rust frontend, speculative-decoding framework and rejection sampler, torch.compile integration. Modified by us via runtime patches and one rebuilt kernel `.so`
vllm-flash-attn (FA3 fork)	github.com/vllm-project/flash-attention	BSD-3-Clause	Source tree for our d9 attention build; we patched `flash_api.cpp`, `static_switch.h`, `tile_size.h` and the V-colmajor code path
DeepGEMM	github.com/deepseek-ai/DeepGEMM	MIT	FP8 block-scaled prefill GEMM (used as-is, extended-K configuration)
FlashInfer	github.com/flashinfer-ai/flashinfer	Apache-2.0	Gated-delta-net chunk kernels (used as-is)
CUTLASS	github.com/NVIDIA/cutlass	BSD-3-Clause	Underlying templates for FA3 and the Humming kernel

Triton	github.com/triton-lang/triton	MIT	Our fastquant, d9 KV writer, n-gram search and ASD acceptance kernels
Humming W4A16 kernel	Third-party W4 GEMM package	As distributed	Kernel headers used as-is; **our contribution is the per-M tile table, dispatch policy and four evaluated kernel variants**
aiperf	github.com/ai-dynamo/aiperf	Apache-2.0	Benchmark client (organiser-specified)
lm-evaluation-harness	github.com/EleutherAI/lm-evaluation-harness	MIT	GPQA evaluation
RULER	github.com/NVIDIA/RULER	Apache-2.0	Long-context evaluation

18.4 APIs and External Services

None. The system runs entirely on the provided hardware. No commercial API, hosted inference service or external endpoint is used at inference time. Model weights and datasets were downloaded from HuggingFace before the benchmark runs.

18.5 Generative AI and AI Coding Tools

AI coding assistants were used during development for code exploration, refactoring, log analysis and documentation drafting. All technical decisions, experimental designs, measurements and conclusions in this report are the work of the team; every number reported here comes from a benchmark or profiler run we executed, and the corresponding artefacts are in the reproducibility package.

19. ORIGINAL CONTRIBUTION DECLARATION

What parts of the submitted solution constitute the team original technical contribution?

- **The FP8 two-level accumulation result - the contribution we would put first.** The decision to measure a trade-off the published data said was not worth testing, and the measurement itself: 2.0-2.5x at the kernel across eight shapes against a control proven bit-identical to the shipped binary on 211/211 kernels, +0.1769 ERS end to end on the competition scale as the dominant component of the attention package, and the head_dim explanation for why it diverges from upstream. Then the audit of what it cost: the chain-length and swamping analysis for head_dim 256; the demonstration that the existing microbenchmarks are structurally incapable of detecting the failure; the GPU-free cost measurement of the upstream fix by static register analysis; the proof that the enable/disable A/B is clean at head_dim 256 because both branches select identical tiles; the adversarial `acc_needle` test with a negative control; and the **periodic scratch-global-memory flush**, which is the mitigation that breaks the register/accuracy trade-off upstream left open.
- The **d9 FP8 attention kernel**: the instruction-level ablation that identified the producer V-transpose and the softmax dependency chain as the two critical paths; the derivation of the exact V permutation; the Triton KV-writer producing that layout; and the four repairs to the vLLM FlashAttention fork without which the column-major path cannot execute at all.
- The **Humming W4A16 tuning and analysis**: the per-M tile table across the speculative-decoding M-range, the Nsight Compute analysis establishing shared-memory bandwidth as the structural ceiling, and four evaluated kernel variants (all bitwise-verified).
- The **Triton FP8 per-token-group quantiser** (2.8-3.0x, bitwise identical in ue8m0 mode).
- The **speculative-prefill proxy**: the text-level proxy architecture in front of an unmodified engine; the `sha256` freezing that preserves prefix-cache hits across turns; the v2.3 query construction and BM25 refill; and the v3 scorer optimisation (255 to 204 ms, byte-identical output on 17/17 regression cases).
- The **sampling-branch adaptation of ASD** with a per-request GPU regret ledger, processed-logit regret (so tokens outside the sampling support can never be accepted), and a bitwise-verified `B = 0` identity.
- The **n-gram drafter overlay** and the lossless argument for it, together with the trace analysis (53% post-EOS steps; 49.7% of mismatches on a single token pair) that motivated it.
- The **serving patches**: `ptail` v2 prefix checkpointing, the GDN CUDA-graph capture and its synchronous-H2D defect fix, the CPU KV offload configuration, and the scheduler policy patches.

- The **experimental methodology**: the noise-floor determination, the controlled minimal-patch ablation ladder, per-request paired comparison, and pre-benchmark configuration verification.

Which components are based on existing or open-source work?

The vLLM v0.28 engine and all components listed in Table 16 and Section 18.3; the FlashAttention-3 source tree; DeepGEMM, FlashInfer, CUTLASS and Triton; the Humming W4A16 kernel headers; the Qwen3.8-27B-FP8, DFlash2 and Qwen2.5-0.5B model weights, all used **unmodified**; and the SpecPrefill and ASD algorithms, whose papers we implemented and adapted (with the adaptations listed above as our contribution).

What is the single most important new technical insight produced during the competition?

That on long-context serving with high prefix reuse, an optimisation must be evaluated by the KV cache it displaces rather than by the compute it saves - and that reducing demand for that cache dominates adding capacity to it. We arrived at this by measuring a *negative* result: speculative decoding, the technique with the strongest published speedups, cost us 8.39 points because its drafter evicted 28.7% of the KV cache and collapsed the prefix-cache hit rate from 82.8% to 43.3%. The same principle then predicted, correctly, that prompt pruning would be worth three times all of our capacity engineering, and explained why a genuine, bitwise-verified attention kernel improvement is worth 17.69 points on a saturated system and nothing measurable on an unsaturated one.

20. REPRODUCIBILITY PACKAGE

The package reproduces the final result from a clean host with two commands.

```
bash serve_btc.sh              # vLLM :8001 + pruning proxy :8000  (about 9-10 min boot)
bash bench_btc.sh <TAG>        # organiser command; writes results/<TAG>/
bash chain_verify.sh <TAG>     # end-to-end: serve -> bench -> paired comparison vs reference
```

Item	Contents
Source code	Serve/bench/verify scripts, all runtime patches, `patch_d9.py`, Humming tile tables, proxy
README	Setup, run, evaluate, reproduce; per-technique READMEs for each contribution
Environment	Container image `vllm/vllm-openai:v0.28.0` plus mount and environment specification; `config.txt` with md5 sums for every patched file
Configuration	Serve flags, environment variables, `cudagraph_sizes.json`, `tuned_lab.json`, proxy parameters
Training scripts	**Not applicable** - no training was performed
Inference scripts	`serve_btc.sh`, `tools/serve_proxy2.sh`, `tools/prune_proxy3.py`
Evaluation scripts	`bench_btc.sh`, `tools/ers_loss.py`, `tools/pair_cmp_btc.py`, RULER and GPQA runners, `quality_ab.py`
Random seeds	20260707 (benchmark, fixed by the organiser); ASD reference tests reproduce `u` via `tl_rand32`
Model access	HuggingFace identifiers in Section 18.2; local paths documented in the README
Experiment logs	Per-run `ers.json`, `profile_export.jsonl`, `server_metrics_export.json`, `prune_stats.json`, `server_docker.log`, `bench.log`, environment dump
Ablation artefacts	One directory per rung K2-K10 plus the baseline, each with the full artefact set
Unit tests	`asd_test.py` (bitwise identity at B=0, no model needed), `test_d9_kvcache.py`, `bitwise_check.py`, `quant_match.py`, scorer determinism suite

An independent evaluator can execute: **Environment setup -> serve -> benchmark -> evaluation -> reproduction of the final result.** Every configuration re-reads `docker logs` and `docker inspect` before benchmarking to confirm the server matches the configuration under test, and speculative-prefill runs additionally probe `/prune_stats` so that a misconfigured run fails in seconds rather than after 25 minutes.

APPENDIX

A. Model Architecture Relevant to the Optimisations

Property	Value	Why it matters here
Layers	64 total: 16 full-attention + 48 gated delta-net	Only 16 layers use the FA3 kernel and the FP8 KV cache
`head_dim`	256	Outside the well-tuned FA3 range; source of the tile and register-pressure problems
Attention heads / KV heads	24 / 4 (GQA)	Determines the decode tile selection (1-MMA-warpgroup path)
`full_attention_interval`	4	One full-attention layer in every four
`max_position_embeddings`	262,144	Requires `VLLM_ALLOW_LONG_MAX_MODEL_LEN=1` to serve at 500k
MLP shapes	gate_up 34816 x 5120; down 5120 x 17408	The two shapes tuned in Table 11
KV cache dtype / block	fp8 / 128 (attention block 1,664)	Block 128 is required by the d9 kernel

B. Decode Step Anatomy (concurrency 1, 60k context)

Measured from an engine trace at the final configuration: **12.8 ms wall, 12.0 ms GPU busy, only 0.84 ms (7%) idle**. The step issues **1,350 kernels, of which 999 are under 10 microseconds** - about 3-4 ms of small-kernel launch tax. Bytes read per step: MLP W4 9.1 GB (3.3 ms), FP8 projections 7.2 GB (2.3 ms), target lm_head bf16 2.5 GB (0.6 ms), drafter bf16 about 1 GB (0.8 ms), FA3 1.2 GB, GDN 1.0 GB. This breakdown is what identifies the remaining decode headroom as *bytes*, not FLOPs, and motivates the future "decode diet" work in Section 16.

C. Additional Ablation Detail

Metric	K0	K2	K4	K5	K8	K9	K10
ERS	.2327	.4096	.4135	.3296	.4028	.5628	**.6511**
s_ttft	.3062	.4633	.4687	.2261	.3122	.4314	**.5404**
s_tpot	.1592	.3559	.3582	.4331	.4933	.6943	**.7617**
Requests	138	231	231	142	393	330	228
TTFT p50 (ms)	927	600	592	5,002	706	710	**531**
TPOT p50 (ms)	31.01	20.55	20.39	14.58	9.02	7.91	**6.38**
Dead TTFT	35	34	33	74	48	44	**14**
Dead TPOT	24	7	8	25	4	4	**0**
Prompt tok/req	108,781	108,163	108,162	110,613	110,906	**66,785**	**66,013**
Prefix hit	78.1%	82.8%	82.8%	43.3%	79.9%	80.0%	**82.8%**
KV computed	22.0%	17.4%	17.4%	56.3%	13.8%	19.2%	**12.4%**
Queue ms/req	815.7	156.6	159.1	3,678	425	198	**55.5**
Prefill ms/req	3,589	1,680	1,683	5,905	1,512	1,130	**748**
Preemptions	0	0	0	11	0	0	0
Tokens/step	1.00	1.00	1.00	2.72	2.87	2.69	2.87
KV cache (tokens)	2,983,076	2,980,456	2,700,325	1,924,636	1,925,019	1,828,997	1,824,024

Table A1. Full server-side metrics for the ablation ladder. The K5 column is the clearest single view of the speculative-decoding regression: every KV-related metric degrades at once.

D. Configuration Verification Signatures

Each ablation rung is confirmed by a distinct log signature read back from the running container, so that no result can be attributed to a configuration the server was not actually running:

Signature in `docker logs` / `docker inspect`	Confirms
`[kpatch] d9: BAT + [d9_kvcache] installed ... V^T pre-permuted (colmajor)`	FA3 d9 attention kernel
`[kpatch] Humming: BAT + [fp8->humming] on: scheme=int4`	Humming W4A16

`group=64`	
`speculative_config=SpeculativeConfig(method=dflash, num_spec_tokens=7)`	DFlash2 speculative decoding
`[asd] installed: B=16 g=0.25 M=2 ... regret=argmax`	ASD acceptance rule
`[offload_fix] ... kv_offloading_size: 143.0`	CPU KV offload
`proxy up (vllm-prune-*, scorer=attn keep=0.2)` + `/prune_stats` returns HTTP 200	Speculative prefill active
`[fix] READY (nodrop=True emit=... h2d=False warmup=False ...)`	Scheduler patches, and that nothing else is enabled